

DDU Fordybelsesprojekt 2023

Projekt robot



Gruppemedlemmer: Anders , Astrid , Mikkell A.
 , Mikkell R. Blom, Rasmus

Vejledere: Kasper , Magnus

Projekt Robot

Udviklingen af S.E.N.N.E.P

Uddannelsessted: Vid Gymnasier HTX Grenaa

Vejledere: Kasper , Magnus

Fag: Digitalt Design og Udvikling

28/11/22 - 12/03/23

Anders

Astrid

Mikkel

Mikkel R. Blom

Rasmus

Abstract

This paper is written as part of a school project and is about the development process of a robot designed to replicate human emotions and needs. It attempts to explore how a robot would be treated if the emotions and mood of the robot could be directly influenced by the people around it. Would it be treated well and be taken care of, or bullied and harassed? During this project there was a heavy focus on making a functional robot that could act independently whilst having it simulate the human emotions ranging from depressed to ecstatic. The group did create a temporarily functional robot with cute aesthetic features as well as a mood system which could be influenced by certain actions committed directly on the robot. Sadly, time combined with technical difficulties proved to be a significant adversary, and it was therefore not possible to conduct a test to evaluate the success of the robot as a social experiment.

Indholdsfortegnelse

Projektbeskrivelse	1
Valg af case	1
Projektstyring.....	1
Udvikling af produkt.....	2
Idégenerering og -udvælgelse	2
Kravmatrix.....	2
Valg af problemformulering.....	3
Den iterative proces.....	3
Koncept	4
Visuelt design	5
Adfærdsdesign	9
Robot Produktion.....	12
Hjul design.....	12
Stel design	19
Komponent opsætning	22
Arduino kode.....	26
Lyddesign	40
Kvalitetssikring	41
Brugertest af robot højde	41
Brugertest af samlet produkt.....	42
Evaluerig.....	43
Konklusion.....	44
Referencer.....	45
Bilag.....	47

Projektbeskrivelse

De fleste af os vil nok hurtigt kunne blive enige om, at det er umoralsk at sparke en hundehvalp. Nogle kan måske til gengæld nikke genkendende til at have slået på elektronik eller andre genstande. Men hvor går grænsen? Hvornår bliver noget "menneskeligt" nok til at det behandles med respekt som en levende ting? Og hvordan får man folk til at føle empati med et livløst objekt?

Valg af case

Vi har i gruppen valgt casen om fysisk interaktionsdesign og kritisk design, hvor formålet er at skabe en spændende, interessant og/eller provokerende brugeroplevelse, der ikke nødvendigvis opfylder kravene om god usability, men i stedet fungerer som kunstprodukt. I arbejdet med denne case er det et krav, at der indgår et eget design af et fysisk interaktionsartefakt.

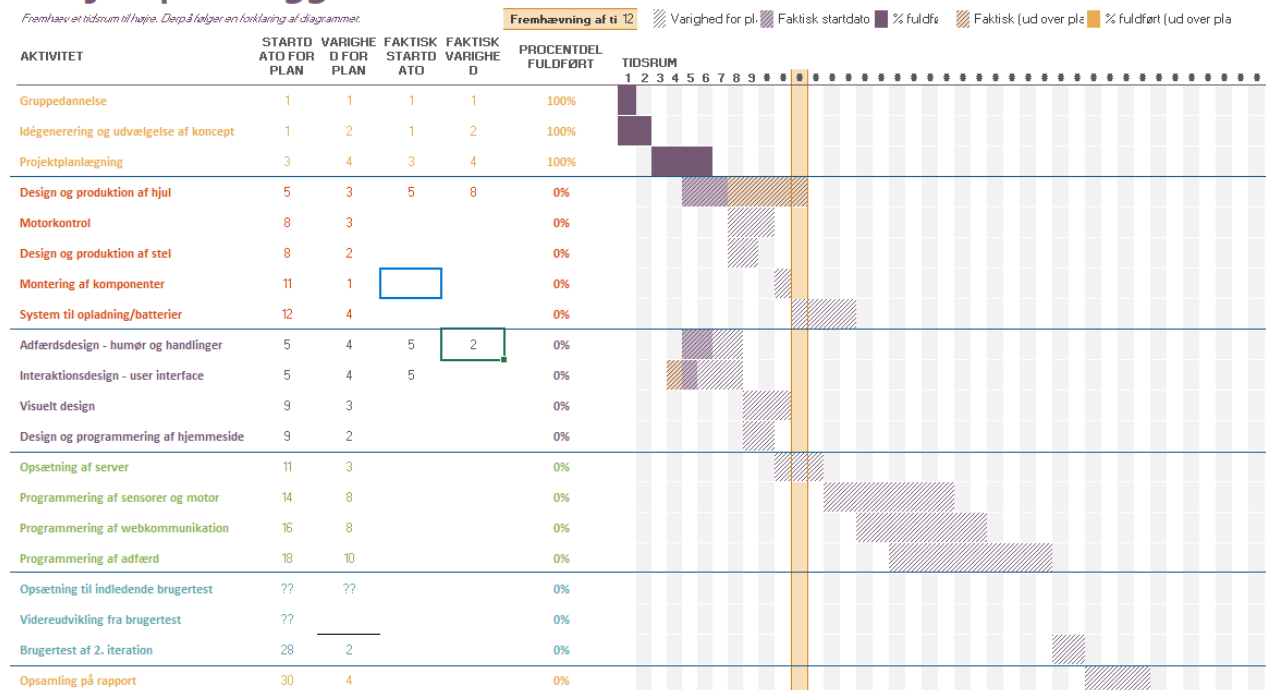
Vi blev i gruppen enige om at vælge denne case, da vi syntes, at det virkede interessant at arbejde med mere kunstnerisk orienteret interaktionsdesign - et område ingen af os havde så meget erfaring med.

Projektstyring

På grund af erfaringer fra tidligere projekter fravalgte vi i gruppen at have fastlagte roller. I stedet rokerede vi rundt mellem opgaverne, alt efter hvor der var mest behov. Formålet var at give en mere dynamisk arbejdsfordeling, da vi i gruppen havde oplevet at fejlvurdere omfanget af opgaver, da vi lavede rollefordelingen, hvilket førte til en ulige fordeling af opgaver. Vi valgte dog stadig at have en projektmanager, der skulle forsøge at holde styr på tidsplanen og sikre klar kommunikation mellem gruppemedlemmerne.

Projektplanlægger

Fremhævet et tidsrum til højre. Der på følger en forklaring af diagrammet.



Figur 1 - Tidsplan

Vi lavede desuden i gruppen en tidsplan, der skulle hjælpe med at visualisere processen, og de delmål vi skulle igennem for at blive færdige. Den blev dog ret hurtigt opgivet, eftersom det viste sig,

at være et meget mere omfattende projekt end vi troede, med endnu flere og tidskrævende opgaver.

Udvikling af produkt

Dette afsnit omhandler hele udviklingen af produktet. Det kommer omkring alt fra de tidlige faser af idégenerering til koden bag robotten.

Idégenerering og -udvælgelse

Efter at have valgt en case lavede projektgruppen en klassisk brainstorm for at finde på idéer til det produkt, der skulle laves i forløbet. Efter brainstormen var der opstået seks idéer, som gruppen valgte at gå videre med. Den første idé var en *grådig slikskål*, idéen var meget simpel, den bestod af en slikskål, som stod frit fremme med slik, som alle kunne tage af, der ville endda være et skilt, hvorpå der ville stå "Tag så meget som du har lyst til". Dog ville der også være en robot arm med et kamera, som fulgte ansigter, og "holdt øje" med hvor meget folk tog, i praksis ville kameraet selvfølgelig være slukket af etiske årsager. Idéen gik simpelthen ud på at se, hvordan et kamera ville påvirke hvor meget slik, der blev taget. *Optisk lysshow* var en mindre provokerende idé og mere fokuseret på noget kunstnerisk. Idéen gik i bund og grund ud på noget lys, som var sat op på en bestemt måde, så det skabte en optisk illusion. En anden simpel idé opstod i form af *glad knappen*. *Glad knappen* gik ud på, at der ville være en kasse med humør knapper, meget i stil med dem der findes i div. butikker, omkring hvor god service brugeren som kunde har modtaget. *Glad knappen* derimod ville ikke omhandle service, men derimod humør. Den ville bestå af et par knapper med forskellige humør niveauer fra deprimeret til ekstatisk. Kassen ville også have et display, hvorpå det ville være muligt at se statistikken for de indsamlede svar. *8-Ball* idéen er meget som det lyder til, det vil være en 8-Ball, dog vil den i stedet for at give ja eller nej og en gang imellem nogle mere vage svar, altid give et opgivende svar som også nogle gange kan være rettet direkte mod brugeren. Den andensidste idé var et interaktivt *rum med lyd*. Et rum som brugeren kunne interagere med og få auditiv feedback. Den sidste idé var at bygge en *robot med følelser*. Robotten ville køre rundt af sig selv og have nogle menneskelignende parametre, som den skal have opfyldt for at være glad. Dens humør ville ændre sig både positivt og negativt, alt efter hvordan den bliver behandlet.

Kravmatrix

For at evaluere på idéerne og udvælge en at gå videre med blev alle idéerne indsat i en kravmatrix (

Krav 1-5	Grådig slikskål	Optisk lysshow	Glad knap	8-Ball	Rum m- lyd	Robot m. følelser
Tankevækkende	4	1	3	2	3	5
Passende ambitionsniveau	3	2	1	1	4	4
Selvstændig brugertest	3	2	5	5	2	4
Passende arbejdsfordeling	1	3	1	2	4	4
Interesse	4	2	1	2	3	5
I alt	15	10	11	12	16	22

Figur 2), og efterfølgende blev kravene vægtet (

Vægtning	Krav 1-5	Grådig slikskål	Optisk lysshow	Glad knap	8-Ball	Rum m- lyd	Robot m. følelser
2	Tankevækkende	8	2	6	4	6	10
1,25	Passende ambitionsniveau	3,75	2,5	1,25	1,25	5	5
1,5	Selvstændig brugertest	4,5	3	7,5	7,5	3	6
1	Passende arbejdsfordeling	1	3	1	2	4	4
1	Interesse	4	2	1	2	3	5
	I alt	21,25	12,5	16,75	16,75	21	30

Figur 3). Hver idé fik tildelt en værdi fra 1-5, for hvert krav, alt efter hvor godt kravet var opfyldt. Værdien 1 betyder "Idéen opfylder kravet meget dårligt", og 5 betyder "Idéen opfylder kravet meget godt". Kravene som idéerne skulle opfylde, var følgende, tankevækkende, passende ambitionsniveau, selvstændig brugertest, passende arbejdsfordeling og interesse. Kravet *tankevækkende* blev vægtet med 2, den højeste vægtning, da den afspejler et krav fra

opgavebeskrivelsen. Ifølge opgaven skal produktet være spændende, provokerende eller lignende. Kravet blev derfor kogt ned til *tankevækkende*, eftersom essensen af opgaven var at skabe noget kunst, der kunne få brugere til at tænke eller overveje noget. *Passende ambitionsniveau*, vægtet 1,25, er et krav, som er med af meget simple grunde, det skal være muligt at lave indenfor tidsrammen, men på samme tid skal det ikke være så nemt, at der ikke er udfordring og derved læring. Det tredje krav *selvstændig brugertest*, vægtet 1,5, kom fra at hele projektgruppen skulle på studietur i en månedstid under forløbet. I den tid ville det ikke være muligt at arbejde videre på projektet, dog var der i gruppen stor interesse for stadig at udnytte tiden. En *selvstændig brugertest* ville kunne gøre netop det, samtidigt med at samle meget data på hvordan produktet som kunstværk ville blive modtaget. *Passende arbejdsfordeling*, vægtet 1 var igen et krav, der omhandlede mere det praktiske omkring udviklingen fremfor selve produktet. Produktet skulle kunne rumme, at alle gruppens medlemmer havde noget at arbejde med igennem hele processen. Sidst men ikke mindst var der *interesse*, vægtet 1, eftersom udviklingen ville forløbe over en længere tidsperiode, blev der vurderet at interesse i produktet fra gruppens side også skulle overvejes. En høj motivation for produktet burde gerne give en bedre arbejdsmoral- og miljø.

Krav 1-5	Grådig slikskål	Optisk lysshow	Glad knap	8-Ball	Rum m- lyd	Robot m. følelser
Tankevækkende	4	1	3	2	3	5
Passende ambitionsniveau	3	2	1	1	4	4
Selvstændig brugertest	3	2	5	5	2	4
Passende arbejdsfordeling	1	3	1	2	4	4
Interesse	4	2	1	2	3	5
I alt	15	10	11	12	16	22

Figur 2 - Screenshot af u-vægtet kravmatrix

Vægtning	Krav 1-5	Grådig slikskål	Optisk lysshow	Glad knap	8-Ball	Rum m- lyd	Robot m. følelser
2	Tankevækkende	8	2	6	4	6	10
1,25	Passende ambitionsniveau	3,75	2,5	1,25	1,25	5	5
1,5	Selvstændig brugertest	4,5	3	7,5	7,5	3	6
1	Passende arbejdsfordeling	1	3	1	2	4	4
1	Interesse	4	2	1	2	3	5
	I alt	21,25	12,5	16,75	16,75	21	30

Figur 3 - Screenshot af vægtet kravmatrix

Valg af problemformulering

Problemformuleringen blev valgt på baggrund af resultatet af kravmatrixen, hvori robotten med følelser havde højest antal point. Idéen med dette produkt er at udforske menneskers reaktion til en robot med menneskelige karaktertræk, specifikt for at undersøge hvordan robotten vil blive behandlet. Vil robotten blive behandlet ordentligt, eller vil den blive sparket? Målet er at skabe en muligvis provokerende, men i hvert fald interessant brugeroplevelse. På baggrund af dette blev den følgende problemformulering skabt:

Hvordan skaber vi et fysisk interaktionsdesign, der undersøger empati for robotter med menneske- eller kæledyrslignende karaktertræk?

Den iterative proces

Iteration er synonymt med ordet gentagelse, den iterative proces er derfor processen af gentagelser. Processen går ud på, at der i løbet af et produkts udvikling vil blive foretaget ændringer, på baggrund af hvad virker og hvad ikke gør. En iteration kan sammenlignes med et stadie, det er enten den nuværende eller tidligere version af et produkt eller også bare en del af et produkt. Alle faser i en produktudvikling kan nemlig gavne af at være iterativ, og det kan hjælpe et produkt meget, at under dets udvikling bliver der testet og vurderet, hvad der virker, og hvad der skal ændres. Ved konstant at revurdere produktet bliver det sikret imod de subjektive holdninger og antagelser, som projektgruppen sidder inde med. Uden den iterative proces vil et produkt hurtigt komme til at

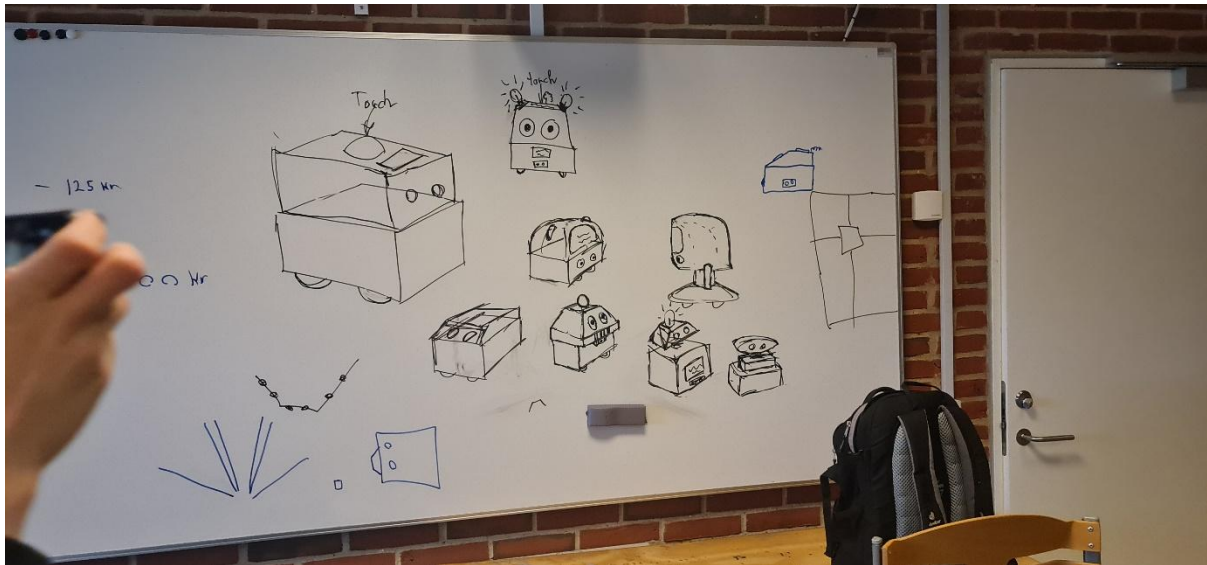
afspejle projektgruppens snæversynet vision, som nemt kan indeholde overseelser i mange områder, f.eks. hvordan andre end produktets egne designere tror, det skal interageres med (Hvass-Raun, Den iterative proces, 2023).

Den iterative proces kan sættes op i en struktur på utallige måder. I en produktion er der mange forskellige processer og emner f.eks. koncept og design, analyse, idé og mange flere. Under en given produktion er der en endeløs mængde af processer og emner, som eventuelt ville kunne tilhøre den iterative proces. Desto mere der tilføjes til den iterative proces, desto mere kompleks bliver den, så det kan med fordel anbefales at holde antallet af emner og processer i den iterative proces på et håndgribeligt niveau så der ikke mistes overblik. Ligesom strukturen kan have endeløst indhold, så kan processen også indeholde endeløse iterationer, der er som regel altid noget, som kan forbedres, hvilket især er sandt for digitale produkter. Det er derfor vigtigt, at produktet på et tidspunkt erklæres for færdigt, eftersom der på et tidspunkt vil være meget lidt forskel på iterationerne, og effekten vil være endnu mindre. Det vil derfor på et tidspunkt ikke kunne svare sig at fortsætte med at udvikle på et produkt (Hvass-Raun, Den iterative proces, 2023).

I dette projekt er der blevet arbejdet med den iterative proces i idéfasen, produktionsfasen og undersøgelsesfasen. I forhold til idéfasen er der begyndt med at idéudvikle og følge som der er kommet nye idéer til, blevet undersøgt og tjekket om hvorvidt det kan svare sig at implementere dem, og om der har været tid. Produktionsfasen er nok den fase med flest iterationer. Der er blevet investeret meget tid på at få et design som virker, uden at gå for meget på kompromis med udseendet. Til sidst med undersøgelsesfasen er der igennem projektet blevet lavet små brugerundersøgelser og prototype tests for at sikre at produktet var på rette spor.

Koncept

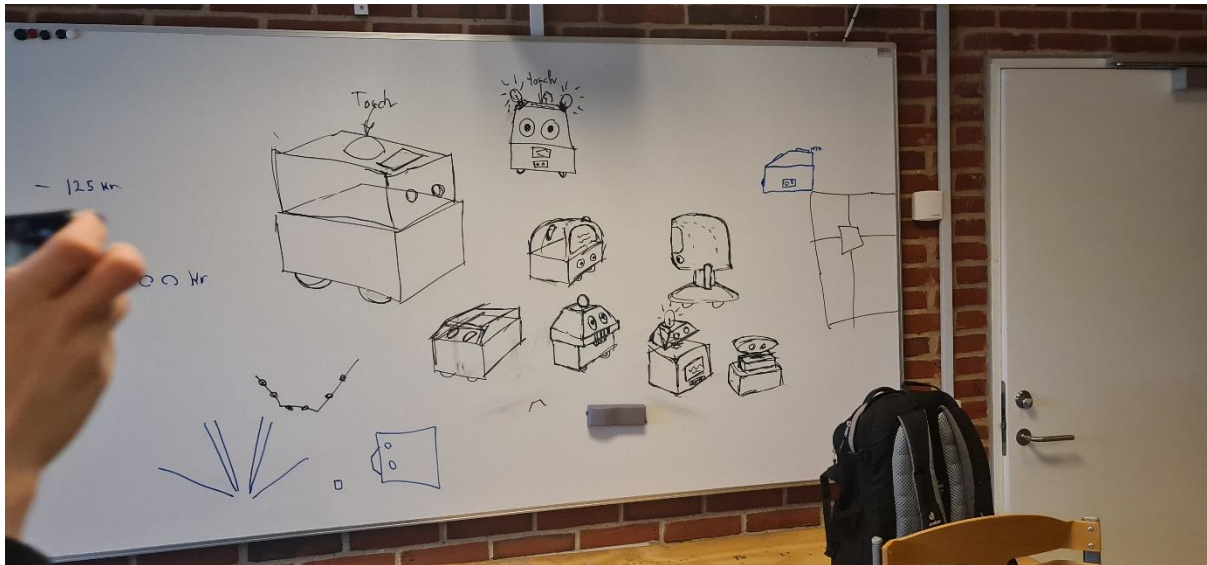
Den udvalgte idé endte med at være robotten med følelser. Efter idéen var valgt blev et need to have, nice to have, skema sat op (Figur 4). Det var selvfølgelig vigtigt at robotten havde de komponenter den skulle bruge til at bevæge sig uden aktiv indflydelse fra et menneske. Den havde også brug for diverse komponenter som gjorde det muligt at interagere med den. Eftersom robotten skulle have følelser, var det selvfølgelig også et krav at der blev udviklet et system til at holde styr på det. Der var også en tanke iblandt gruppens medlemmer at den skulle have adgang til en server som kunne håndtere både dens humør og handlinger, hvilket vil blive forklaret mere i dybden senere. Nogle af de ønsker der var omkring hvad robotten ellers kunne have var diverse behov, såsom energi, social og humør for at efterligne et menneskes behov. Der var også et ønske om manuelt at kunne styre robotten med et eksternt apparat, samt en hjemmeside hvorpå det ville være muligt at se dens humør og de interaktioner den havde foretaget. Hjemmesiden ville hovedsageligt være til dokumentation, men også så gruppen kunne følge med under den længerevarende brugertest, der også var en forhåbning om at ville finde sted. Eftersom æstetik også var en prioritet for at tiltrække opmærksomhed, og derved interaktion blev der tegnet flere koncepttegninger med forskellige grader af sød og praktisk (



Figur 5).

Need to have	Nice to have
- Omnidirektionelle hjul - Funktionelt og æstetisk stel - Arduino med wi-fi - Ultrasoniske dybdesensorer - Accelerometer - Touch display med information, og til at nusse robotten - Server der kan styre humør og handlinger - Humørvariabel der påvirkes af interaktion - Algoritme til at lave handlinger ud fra humør - Højttaler og lyde der viser humør - Sluk/tænd timer - Indsamle data over humør og interaktioner og opbevare på serveren	- Forskellige behov: energi, social, humør - Controller override mode til manuel styring - Hjemmeside med logbog over humør og interaktioner

Figur 4 - Skema over need to have og nice to have



Figur 5 - Tidlige skitser af robotten

Visuelt design

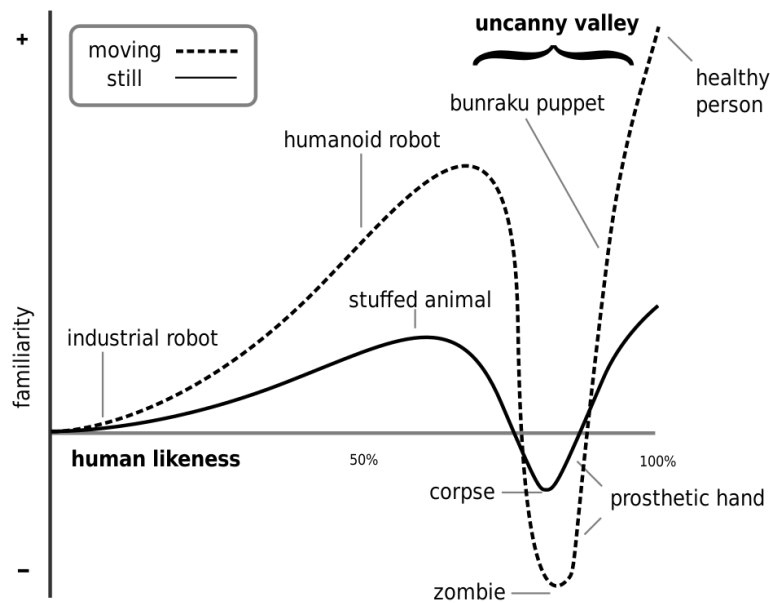
Efter konceptudviklingen blev vi enige om, at vi ville forsøge at opnå et visuelt design, som gav indtrykket af, at robotten var "cute", som gav indtrykket af at være levende, men med et ønske om at undgå uncanny valley fænomenet.



Figur 6 - Bemærk hvordan bjørnebamsens proportioner (Midterst) minder mere om et barns end en bjørns. Bl.a. kan dette ses ved det store hoved og proportionerne for længden på lemmerne ift. resten af kroppen.

Under normale omstændigheder skaber bekendthed med et objekt eller fænomen tryghed. Dette forklarer, hvorfor antropomorfering til en vis grad giver et indtryk af, at noget er sødt. Derfor tegnes dyr i børneshows ofte med menneskelignende ansigter, og den klassiske bjørnebamses anatomi er langt mere barnelignende end bjørnelignende (Figur 6). Dog opleves et fænomen også,

hvor noget bliver uhyggeligt grundet dets lighed med mennesket.

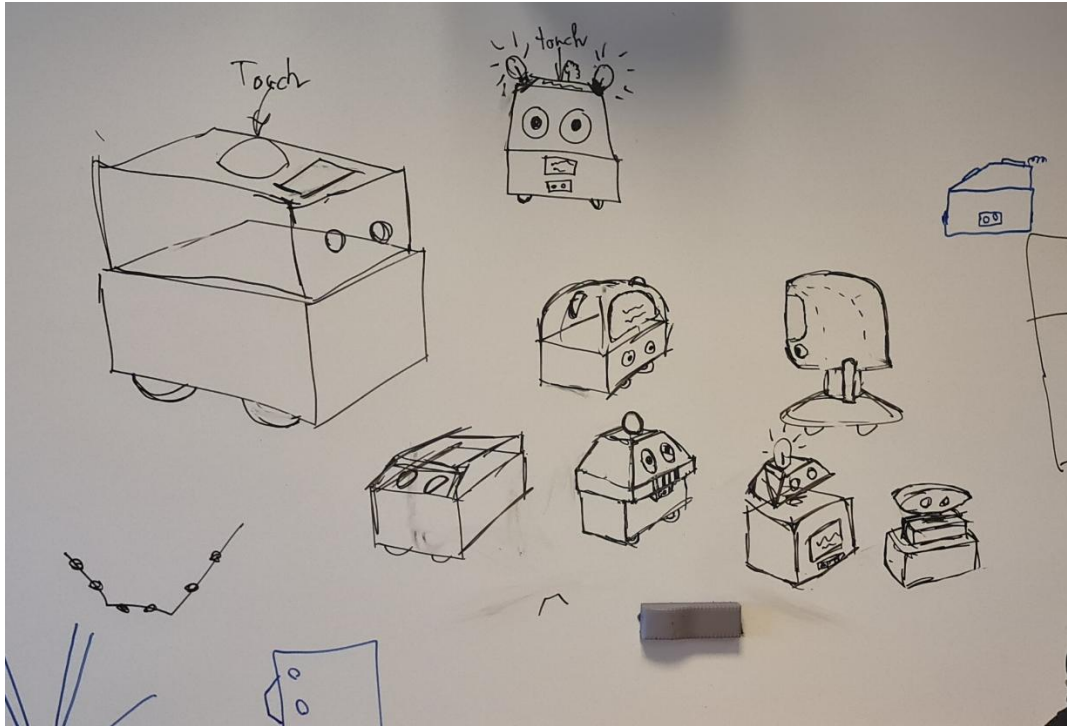


Figur 7 - Emotionel reaktion plottet mod antropomorfi. (Mori, 2012)

Dette fænomen kaldes "Uncanny valley" (Figur 7). En forklaring på dette kunne være, at mennesker knytter noget dødt men menneskelignende til at være et lig, og at det har været en evolutionær fordel at holde sig væk fra lig, da de potentielt har kunne have været inficeret af sygdomme. (Mori, 2012)

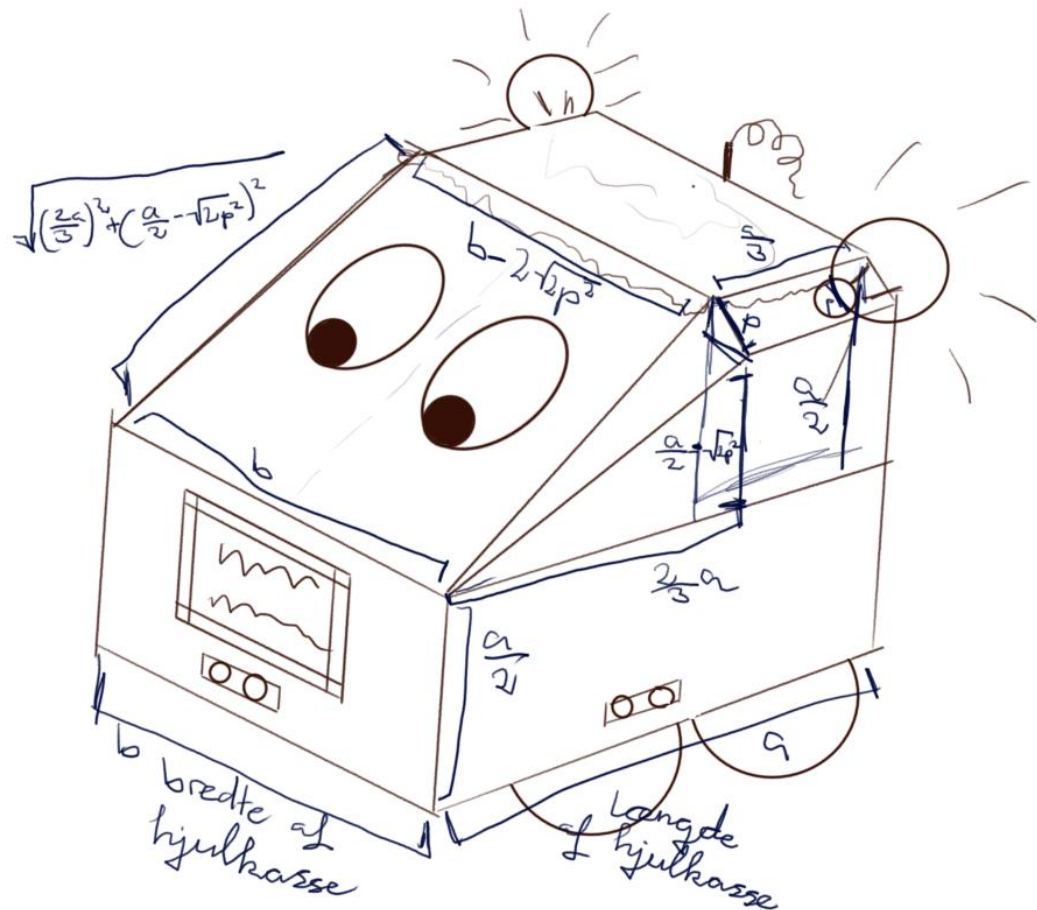
Yderligere er det teoretiseret, at fornemmelsen af at noget er sødt stammer fra dets lighed med spædbørn, da det også har været en evolutionær fordel at tage sig af små børn. Disse klassiske træk der gør noget sødt, inkluderer derfor ofte store hoveder, store øjne, og at være lille. (Smith, 2018)

Grunden til valget af, at designet skulle være sødt, var for at øge brugerens empati for robotten. Dette ville også give brugeren et større emotionelt respons, når robotten fx havde humørstadiet "deprimeret", og på denne måde også give brugeren et større incitament til at interagere med den. Designet blev derfor lavet med bagtanken, at det skulle give brugeren et indtryk af, at den havde et stort hoved og store øjne, men at den ikke måtte være mere menneskelignende end dette, da fx at give den en menneskelig form ville risikere at ramme uncanny valley. Der var dog også kriterierne, at robotten skulle indeholde en skærm, have en touchsensor et brugervenligt sted, samt have plads til ultrasoniske sensorer.



Figur 8 - Billede fra brainstorm til visuelt design.

Der blev tegnet en række idéer til potentielle visuelle designs, hvor vi i gruppen traf en beslutning om at tage udgangspunkt i det øverste tegnet på tavlen (Figur 8). Dette var med udgangspunkt i en kombination af hvor sødt vi synes de forskellige designs var, men det var også grundet at det var teknisk simplere at bygge end mange af de andre designs. Yderligere var der her plads til sensorer i bunden af kassen.



Figur 9 - Rentegning af koncept, samt forsøg på at finde byggemål.

Det endelige designs intention var, at slører hvor grænsen mellem krop og "hoved" gik på robotten, og på denne måde give et indtryk af et stort hoved. Yderligere skulle robotten have store googlie eyes, da dette igen referer til spædbørn. Dog uden et fuldt ansigt, da dette ville være uncanny, men i stedet med en skærm på frontsiden, for at brugeren kan læse dens "tanker". Yderligere var der på dette tidspunkt også en idé til, at der skulle være store pærer på siden af robotten, som også ville kunne bruges som emotionelt respons i forskellige situationer ved bl.a. at blinke, være tændt, slukke, o.l. I denne version af designet er der også inkorporeret skrå sider rundt om robotens "ansigt", da dette giver et blødere visuelt indtryk end skarpe sider. (Figur 9)



Figur 10 - Endelig robot, 3D model.

Fra det endelige koncept til den reelle robot skete en række ændringer af praktiske årsager. Skærmen blev flyttet op på toppen af robotten, da dette var mere brugervenligt end på siden. På denne måde kan brugeren læse dens humør samt beskeder stående fremfor at skulle sætte sig på hug. Yderligere blev portionerne på robotten ændret, da hjulene fyldte mere end først forudset. Hvis proportionerne skulle have forblevet de samme, skulle robotten have været noget højere end først forudset, og derved skulle den bære mere vægt end forudset. Yderligere blev skråsiderne fjernet, da dette viste sig var svært at bygge i praksis. Lysene blev også fjernet, da dette havde krævet flere pins i arduinoen, og da det var en højere prioritet at få inkorporeret på de resterende komponenter og sensorer. (Figur 10)

Adfærdsdesign

I arbejdet med at lave robotten er et af designpunkterne interaktions- og adfærdsdesign - altså både hvordan brugeren interagerer med robotten, men også hvordan robottens autonome adfærd udspiller sig. Robotten skal kunne vælge og udføre handlinger tilfældigt fra en liste af muligheder, der er vægtet ud fra dens humør.

Robotten skal også udvise en vis grad af personlighed for at skabe empati og relation hos brugeren. Dens udtryk og adfærd skal designes til at minde om en nysgerrig (og lettere godtroende) hundehvalp; da dette er en type personlighed brugeren kan genkende og forhåbentlig synes er sød at interagere med. Derudover skal brugeren kunne læse dens tanker i form af citater på displayet, for at gøre robotten mere menneskelignende og give et indblik i dens følelser.

For at skabe overblik over robottens adfærd, opstillede vi i gruppen et par skemaer der fungerede som oversigt over de tilfældige handlinger, vi ville sigte efter at få robotten til at udføre:

Navn	Beskrivelse
Efterforsk	Robotten kigger rundt efter objekter i bevægelse og bevæger sig i retning af bevægelsen, hvis den ser noget. Derudover afspiller robotten en nysgerrig lyd. Hvis ikke den ser noget der bevæger sig, kører den bare i en tilfældig retning.

Sang	Hvis robotten er i meget godt humør, kan den finde på at afspille en lille sang og køre rundt.
Lur	Robotten finder et hjørne (hvis muligt?) og afspiller en snorkende lyd. Anden adfærd deaktiveres i et par minutter. Hvis nogen kommer tæt på, vågner den.
Snurre rundt	Robotten snurrer rundt om sig selv for sjov. Hvis den gør dette for meget, bliver den rundtosset og kører i zigzag.
Opmærksomhed	Robotten laver en gentagende lyd og tigger om opmærksomhed (evt. via displayet)
Selvhad	Når robottens humør er lavt, kan den finde på at bevidst køre ind i vægge, hvorefter den vil bakke og forsøge at køre ind i væggen igen.
Vrede	Robotten forsøger bevidst at køre med fronten ind i folk mens den laver vrede lyde.
Nysgerrig	Robotten søger imod høje lyde og spiller en nysgerrig lyd.
Undvig	Robottens tærskel for at flygte fra potentielle kollisioner stiger jo lavere dens humør er. Ved de lavere stadier af humør kan den finde på at gå i panik, hvor den laver bange lyde og flygter i længere tid end den plejer. Dette er også tilfældet for høje lyde.
Berøring	Hvis robotten bliver berørt på touch-displayet laver den en tilfreds lyd og dens humør stiger.

Events og citater efter humør - øverste række beskriver intervallet som handlingerne sker i, og tallene ved siden af hver handling beskriver vægtningen af den handling når robottens humør er indenfor intervallet:

0-15	16-30	50-70	71-85	86-100
Efterforsk (50) Selvhad (25) Vrede (25) Undvig (*)	Efterforsk (60) Lur (15) Selvhad (25) Undvig (*)	Efterforsk (45) Nysgerrig (45) Lur (10)	Efterforsk (30) Lur (5) Snurre rundt (20) Nysgerrig (45)	Sang (15) Efterforsk (25) Lur (5) Snurre rundt (15) Nysgerrig (30)
"Jeg Hader Industrisamfundet." "Jeg Dræber Mig Selv" "Jeg Føler Mig Håbløs Og Ulykkelig." "Jeg Kan Ikke Se En Positiv Fremtid." "(X_X)" "(ಥ_ಥ)" "Fodr Mig Små Dyr" "..." "Kan Man Dø Af Det Her" "Giv Mig Din Nyre"	"Jeg Savner Min Far (Mikkel Blom)" "Hvorfor Er Jeg Her" "...Bip" "(☹_☹;)" "Jeg Er Træt Af At Føle Mig Uforstået" "Dør Herovre LOL" "Det Her Sted Er Rædsomt" "Definitionen Af Kedsomhed:" "Hhhhhhhhhhhhhhh" "Jeg Har Også Følelser"	"Hej Med Dig" "Vil Du Nusse Mig På Hovedet ?" "Jeg Keder Mig Lidt" "Hvad Er Dette Sted" "Tror Du På Gud" "Hvornår Er Der Mad" "Mit Navn Er SENNEP ... Hvad Er Dit ?"	"Jeg Nyder At Udforske Skolen" "^(^_^^)" "Jeg Har Det Sjovt" "Hihi" "Mine Tandhjul Summer Af Glæde" "Skal Vi Til Fredagscafé" "Hvad Betyder Mucki Bar"	"(☺'☺☺)" "q(≧▽≦q)" "Hej Til Alle Mine Venner" "Jubii !!!" "Jeg Elsker Livet På Skolen" "Hejsa (☺☺☺)" "Intet Kan Stoppe Mig!" "Jeg Er Uovervindelig!"

Robot Produktion

Her vil der blive gennemgået produktionen for hjulenes design, produktionen af stellet samt komponentliste. Produktionen gjorde stor brug af iterativ proces, og da robotten potentielt skulle kunne modstå fysisk vold, var det vigtigt i produktionen at den var holdbar.

Hjul design

I dette afsnit beskrives teorien bag de hjul vi benytter til robotten, samt de valg der er foretaget i designet og produktionen af hjulene.

Teori

En af de vigtigste elementer ved robotten, er dens hjul. Uden hjul, kunne den ikke bevæge sig. Derfor er det også vigtigt at benytte det bedst mulige design for hjulene, så robotten fungerer optimalt. Der er flere faktorer, som er relevante, når det kommer til hjul design.

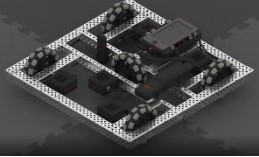
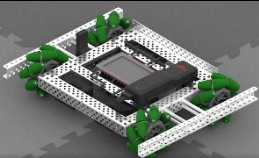
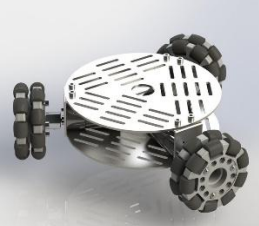
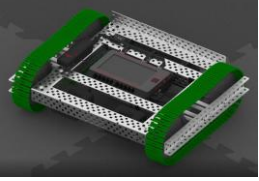
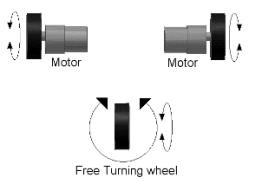
Manøvredygtighed er en af de væsentligste problematikker, som kan løses ved et godt hjul design. Manøvredygtighed dækker over mobilitet og styrbarhed. Altså, evnen til at bevæge sig, og evnen til at blive styret. Mobilitet er vigtigt for at robotten kan bevæge sig til en eftertragtet position. Hvis bevægelsen er langsom eller upræcis, vil det være svært at foretage handlinger baseret på omgivelserne. Eftersom at robotten skal kunne undgå vægge, objekter, mennesker, og andet, skal den kunne reagere hurtigt, før den rammer den givende forhindring. Reaktionstiden påvirkes til dels af sensorerne, samt softwareudregningerne, men også af hjulets parametre. Herunder hjulets acceleration, deceleration, friktion, robottens vægt, og mange andre parametre, der hidtil stadig var ukendte.

Derudover er det også centralt, at robotten kan styres præcist, for at den f.eks. ikke sætte sig fast i hjørner, tætte gange, og andet. Eftersom at robotten vil blive udsat for dynamisk ændrende omgivelser, med mennesker der bevæger sig, objekter der kan flyttes eller roteres, eller dens egen kode, der kan slå sensorerne fra, og essentielt gøre den blind. Dette uforudsigelige miljø giver grundlag for et styresystem med hjul, som ikke mindst er hurtige og præcise, men også med holonomisk bevægelse. Holonomisk plan bevægelse betegnes som enten "[...] Any robot [...] that allows for direct [...] motion along the x and y axes" (MIT Computer Science & Artificial Intelligence Laboratory, 2001) eller "[...] any robot that can perform arbitrary rotations and translations" (MIT Computer Science & Artificial Intelligence Laboratory, 2001). Dvs. at en robot betegnes som holonomisk, hvis den enten kan bevæge sig direkte i enhver retning på xy planet, eller, hvis den kan rotere omkring sin egen akse. Denne form for styring giver mulighed for at reducere antallet af situationer, hvor robotten kan sætte sig fast.

Forskellige typer hjul-drev

For at opnå denne præcise og hurtige styring, kan der anvendes et væld af forskellige hjul-drev, hver med deres egne fordele og ulemper. Eftersom der er et stort antal forskellige hjul-drev, har vi udvalgt og sammenlignet nogle af de mere velkendte og dokumenteret hjul-drev. Dette er for at sikre, at det bedste hjul-drev bliver benyttet.

Navn	Beskrivelse	Billede	Fordele	Ulemper
------	-------------	---------	---------	---------

H-drev	Benytter fem standard omni-hjul, hvor fire af dem er opstillet som hjulene på en bil, og det sidste hjul er placeret på tværs i midten af robotten.	 (vexrobotics, 2023)	Simple hjul, god stabilitet, holonomisk bevægelse	Kræver mange dele, skal være forholdsvis stor, kræver komplekst stel
Mecanum-drev	Benytter fire mecanum-hjul, som er opstillet ligesom hjul på en bil	 (vexrobotics, 2023)	God stabilitet, holonomisk bevægelse, simpelt stel, relativt kompakt	Kompliceret hjul, kræver mange dele
3 hjul holonomic-drev	Benytter tre omni-hjul, hver adskilt 120 grader og vender vinkelret mod midten af robotten	 (Trinh, 2018)	Kompakt, kræver få dele, simple hjul, holonomisk bevægelse	Mindre stabilt, besværlig konstruktion (Runde dele)
4 hjul holonomic-drev	Benytter fire omni-hjul, hver adskilt 90 grader og vender vinkelret mod midten af robotten	 (vexrobotics, 2023)	Simple hjul, god stabilitet	Kræver mange dele, skal være forholdsvis stor, ikke holonomisk bevægelse, komplekst stel
Larvefødder	Benytter to bånd, hver forbundet mellem forreste og bagerste hjul	 (vexrobotics, 2023)	Simple hjul, god stabilitet, simpelt stel	Kræver mange dele, skal være forholdsvis stor, ikke holonomisk bevægelse
Differential drev	Benytter to standard hjul placeret bagere på hver sin side, samt et svinghjul i midten foran.	 (Wikipedia, 2023)	Simple hjul, kompakt, kræver få dele, simpelt stel	Ikke holonomisk bevægelse, Mindre stabilt

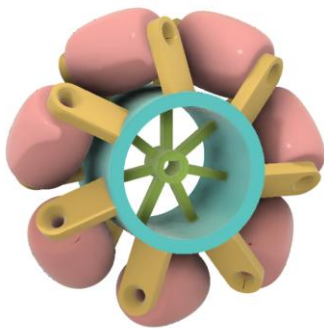
Som tidligere nævnt, var reaktionshastighed, manøvreedygtighed og præcision de vigtigste elementer der blev overvejet, alle tre påvirket af vægt, størrelse og stabilitet. Derudover er der tidligere lagt grundlag for at en robot med holonomisk bevægelse er essentielt. Vi har derfor, på baggrund af tabellen, valgt mecanum drev, pga. dens positive ratio af fordele til ulemper (Se tabellen ovenfor).

Det er på baggrund af denne viden, at første iteration er blevet udformet.

Iteration 1

I første iteration, er der ikke overvejet variabler som hjulenes friktion, motorkontrol, eller andet. Denne iteration var med til at skabe overblik hjultypen der bliver brugt, samt skabe grundlag for hvilke problematikker der er ved designet af robotten. Ud fra teorien om holonomiske plan bevægelser, samt hjul-drev typerne, blev valget reduceret til et mecanum drev. Denne iteration var til for at skabe en hurtig prototype af et mecanum hjul, for at udforske dens teori og mekanik. Den fysiske prototype af hjulet, var med til at afvikle flere tidlige fejl ved designet. Herunder, størrelsen af rollerne, bredden af hjulet, friktionen af rollerne, osv. Dette første hjul var ikke tilsigtet, at det skulle bruges som hjul, men det skulle øge vores viden om et hidtil relativt ukendt emne.

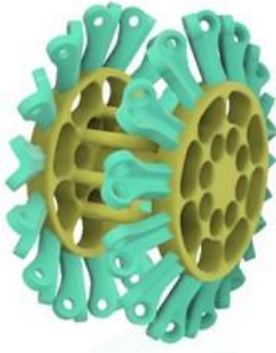
Under vores indledende interaktioner med det fysiske mecanum hjul, blev vi mere velkendte med fysikken bag hjulet, hvori det også blev tydeligt at vi ikke havde valgt det forkerte hjul. Ved denne prototype sikrede vi os, at det faktisk var muligt for os, både at lave et sådan hjul, men også bruge det i vores projekt. Hvis ikke vi havde gjort dette til at starte med, kunne vi have endt med nogle hjul, der i sidste ende ikke ville virke. Som nævnt tidligere, så er dette hele essensen bag den iterative proces, og det endte med at reducere vores spildtid væsentligt. Nedenfor ses en 3d model af den første prototype (Figur 11)



Figur 11 - Screenshot af 3D modellen til første iteration

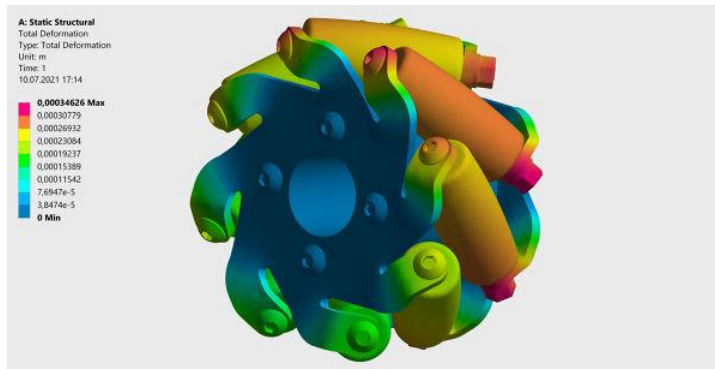
Iteration 2

I anden iteration eksperimenteres der med at få implementeret alle de ændringer og fejl der blev fundet frem til i første iteration. Der prøves at holde vægten nede ved, blandt andet, at lave huller i hubben (midten) (Figur 12, Specifikt det grønne komponent).



Figur 12 - Screenshot af 3D modellen til anden iteration

Hullerne bliver placeret der, hvor det ikke er nødvendigt at have 100% filament. Eftersom at det er rollernes arme, der modtager størstedelen af stresset, som kan ses nedenfor i en Static Structural FEA (Finite element analysis) (Figur 13). Test softwaren benyttet i analysen, er Ansys Workbench (Ansys, 2023), som i deres egne ord "[...] provides quality products with reliability [...] and be assured of accurate results" (Ansys, 2023). Resultatet viser, at størstedelen af stresset på hjulene, går til roller armene.



Figur 13 - FEA Stress test af mecanum wheel (Payne, 2021) Rød er den største stres og deformation, og blå er der hvor der er mindst stress og ingen deformation.

Derudover blev der også midten af hjulet og reduceret til et par stolper. Disse reduceringer var med til at mindske vægten af hvert hjul, eftersom vi på dette tidspunkt i processen, stadig ikke vidste hvor tung robotten ville være, hvilken vil have en effekt på hastigheden og manøvreedygtigheden af robotten. Derudover vil vægten af robotten også have effekt på valget af motor, der skulle køre robotten. Med disse ukendte faktorer, var det ikke muligt at producere stellet af robotten, før hjulene var designet.

Andre ændringer i denne iteration, er roller armene, som før hen kun understøttede én retning hvorpå rollerne kunne sættes på. Armene er nu blevet ændret til et "dual-design" (Figur 12, Specifikt de lyseblå komponenter) (Se også Figur 11, Specifikt de gule komponenter, for sammenligning), hvor rollers kan sættes på, på begge led, så der kunne 3D printes fire identiske hjul, frem for at printe separate hjul til hver af de fire hjul. På grund af den specifikke opsætning af hjulene, var det nødvendigt at ændre retningen af rollerne, på nogle af hjulene. Dette bliver yderligere forklaret under "Komponent opsætning -> Motor control"

Yderligere, er størrelsen blevet øget på modellen, eftersom den første prototype viste sig at være for lille til effektivt, at køre rundt med. Samt er antallet af rollers blevet øget. For konstant at have en roller til at røre jorden, skal selve roller tykkelsen øges, eller også skal der tilføjes flere rollers. Eftersom bredden af rollerne ikke kunne øges særlig meget, ud fra længden af armene, skulle der enten tilføjes flere arme eller så skulle armenes længde øges. Vi valgte at øge antallet af arme, eftersom det havde flere fordele. Det gjorde, at vi konstant havde en roller i kontakt med jorden, samtidig med at hjulets bevægelse blev mere blød. Det havde samme effekt som hvis et firkantet hjul blev udskiftet med et ottekantet hjul. Jo flere rollers, jo tættere kommer hjulet på en cirkel, ergo, en bløder bevægelse.

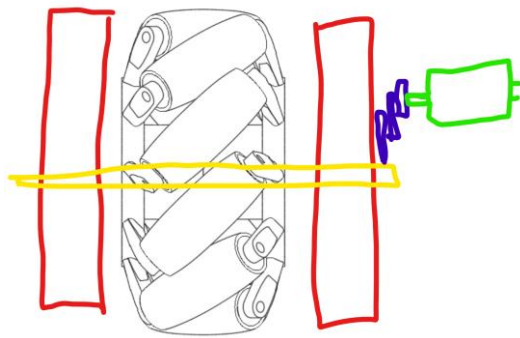
Iteration 3

Viden vi fik fra sidste iteration, er blevet indført. Vi printede rollerne i PLA-filament og fandt ud af, at de ikke havde særlig god trækraft og friktion med gulvet. Dette er blevet løst ved at printe et ydre lag af gummi-filament til rollerne, hvilket gav meget bedre trækraft. Se Figur 14, specifikt det mørkeblå komponent. Her kan ses det ydre lag til det grønne komponent (rolleren).



Figur 14 - Screenshot af 3D modellen af en roller (Grøn) og dens ydre lag (Blå)

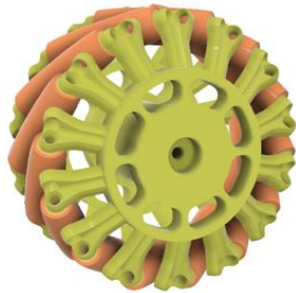
Derudover skiftede vores fokus fra at udforme designet til hjulet, til at gøre hjulet klar til robotten. Det overordnet hjuldesign var udformet nok, til at vi kunne begynde at overveje hvordan hjulet skulle benyttes af en robot, samt gå mere i dybden med de mindre dele. F.eks. overvejede vi hvordan rollerne skulle fastmonteres til selve hjulet. Det blev overvejet at benytte maskinskruer, der gik gennem rollerne, som sættes sammen med møtrikker i hver ende. Dog var det ikke muligt at finde maskinskruer, der var lange nok, så i stedet valgte vi at gå med et 4mm Ø gevindstang, skåret ud til at være ca. 65mm lang. Det samme blev overvejet til at sætte hjulet fast til robotten. Her brugte vi et 5mm Ø metalrør (Gul), som skal sidde fast til stellet af robotten (Rød), så hjulet kunne roteres af en motor (Grøn), gennem et gearsystem (Blå) (Figur 15).



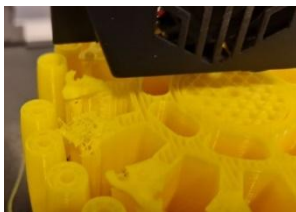
Figur 15 - Skitse af ideen til at køre hjulet med metalpind og gearkasse

Derudover er der lavet et større hul i midten af hjulet (Figur 16, specifikt midten af det grønne komponent) til at fastmontere en ringklemme, til at sætte metalrøret fast. Denne iteration har også gjort os opmærksom på, at det ikke er pålideligt at 3D-printe hele hjulet, som vi tidligere har gjort. Eftersom at hjulet krævet et meget præcist og detaljeret print, har vi hidtil kun printet hjulet som en helhed, da det sikrede os, at alle armene og delene sad godt sammen. Men dette var ikke længere som en reel mulighed, fordi hvert hjul kunne tage op til flere dage (2 dage og 9 timer), at printe. Derudover har printet også en stor tendens til at mislykkes (Figur 17), hvilket leder til at hele printet skal laves om. Det blev derfor for dyrt, tidskrævende og upålideligt, at fortsætte med at printe på denne måde. En løsning på problemet, var at reducere kompleksiteten af hjulet. Det blev gjort ved at dele hjulet op i 4 dele. Der er midtersøjlen, som har lav tendens til at mislykkes, da det ikke er en særlig kompleks form, eftersom den ingen "overhangs" har. Dernæst er der hver af siderne for sig, men med små indhak og pinde, hvor rollerne skal være. Og til sidst, er der selve armene (Figur 18, Specifikt det grønne komponent, for at se indhakked og pinende, og det lyseblå komponent for armene). Denne ændring skulle gøre det muligt at printe roller armene for sig selv, og lime dem på efter. For at teste teorien, blev der printet en lille sektion af siden med indhakked, samt en arm, som blev limet sammen. De blev så udsat for en stress test, hvor vi prøvede at knække dem fra hinanden (Figur 19). Testen fungerede i lille skala, men da vi skalerede det op til et fuld hjul, var der ikke meget

sparet tid. Derudover sad armene ikke nær så godt fast, som i testen. Denne ide blev derfor hurtigt udfaset.



Figur 16 - Screenshot af 3D modellen af tredje iteration



Figur 17 - Billede af et 3D print der gik i stykker under print

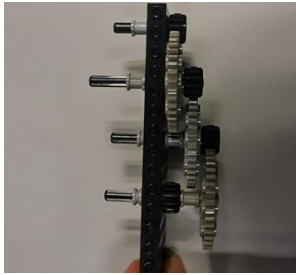


Figur 18 - Screenshot af roller armene separat fra hjulsiderne, samt indhak og pind



Figur 19 - Billede af "knæk"-test af sammenlimet arm og hjulside udsnit

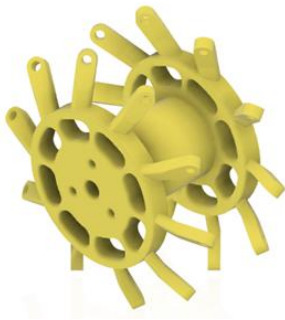
Den sidste der blev eksperimenteret med i denne iteration, var at bruge et tandhjul til at lave en "gearkasse" til hvert hjul (Figur 20). Dette var gjort, for at gøre det muligt for robotten at flytte sig hurtigt, samt stadig kunne gøre brug af motorer vi allerede havde tilgængelige. F.eks. blev der eksperimenteret med at benytte det store hul i midten af hjulet (Figur 16, specifikt midten af det grønne komponent), til at sætte et tandhjul fast, så motoren kan køre en tandrem til at få hjulet til at køre rundt. Denne ændring blev dog aldrig indført, eftersom vi ikke kunne købe eller producere tandhjul, der var små nok til at virke effektivt, samtidig med at passe ind i robotten.



Figur 20 - Billede af lego-testopsætning af gearkasse system

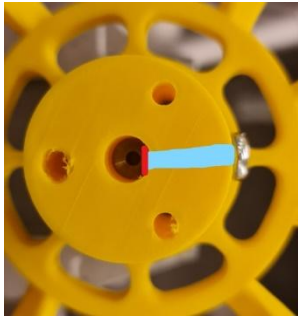
Iteration 4

I stedet for at bruge en gearkasse, vendte vi tilbage til det første "test" design, hvor hjulene kun kunne sættes direkte på en motor. Derudover gik vi også tilbage til at bruge roller arme, der ikke understøttede begge retningsmulighederne (Figur 21). Dette blev gjort grundet komplikationer i det tidligere "doubleholder" -design. Eftersom support strukturen var så omfattende, blev det nær umuligt at fjerne supporten uden samtidig at brække de relativt skrøbelige holderarme. Derudover gjorde doubleholder-designet også at printtiden blev omfattende, hvilket ikke var en mulighed på dette tidspunkt i produktionen. De mange komplikationer havde ført produktionsfasen langt over tid, hvilket gav grundlag for en ændring i taktik. Fokuset blev flyttet fra at lave det mest optimale hjul, til faktisk at få lavet et hjul. Denne ændring medførte nogle kompromisser. For det først blev holder ændret, som nævnt tidligere. Denne ændring medførte dog at hjulet blev mindre optimalt, da hjulet nu ikke var så universelt brugbart som det kunne have været. Derudover blev hjulet opdelt i to stykker, altså den blev delt på midten. Flere opdelingsmuligheder blev overvejet, men denne mulighed blev valgt på baggrund af printtiden, integriteten af hjulet, antallet af dele, og kompleksiteten.



Figur 21 - Screenshot af 3D modellen til fjerde iteration

Derudover blev der lavet yderligere to tilføjelser til hjulet, før det blev printet. Der blev tilføjet et hul i toppen af hjulet, der blev boret med et gevindbor, hvor en 12mm M4 maskinskruer kunne sættes fast. På Figur 22, kan det ses hvordan skruen (lyseblå) går gennem hjulet, og sætter sig fast på den flade overflade af motorens skaft, hvilket låser hjulet fast til motoren. Derudover er det tilføjet tre huller, spredt rundt om cylinderen, som er med til at samle de to sider af hjulet. Eftersom, at hvert hjul blev printet i to dele, var det nødvendigt at sikre at hjulene blev sat sammen præcist, for at rollerne passede. Dette blev opnået, ved at lave huller til tre gevindstænger, som blev sat fast med møtrikker på begge sider af hjulet (Figur 22 og Figur 23). Derudover blev delene også limet sammen, samt blev alle møtrikker på hjulet behandlet med loctite skruelåsning, for at forhindre at møtrikkerne blev løse under kørsel.



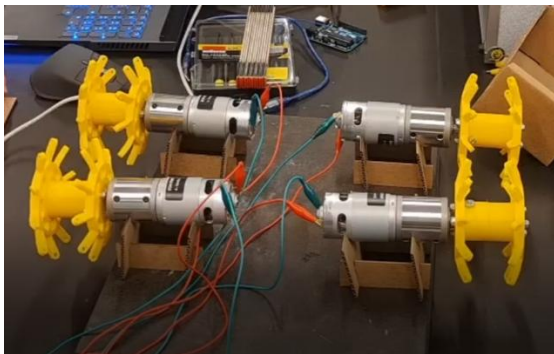
Figur 22 - Billede skitse af hjulmonterings mekanismen



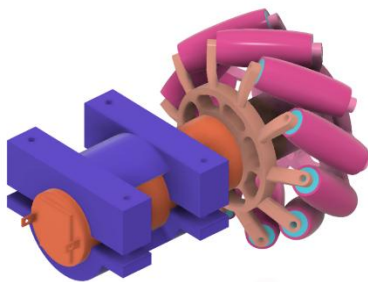
Figur 23 - Billede af hjulet under produktion

Stel design

Efter hjulene var designet, var det meget nemt at forme stellet omkring dem. Det første problem der opstod, var at motorerne skulle sættes fast til stellet (Figur 24). Normalt ville der bruges et "motor mount" til dette, men eftersom vi ikke havde nogen i den størrelse vi skulle bruge, valgte vi at designe vores egne (Figur 25). Denne mount gjorde det muligt at sætte motorerne fast til bundpladen, med gevindstænger og møtrikker. Dog var der et problem med dette design. Hvis hjulet blev blokeret, ville motoren dreje rundt om sig selv. Dette blev fikset ved at stramme møtrikkerne, så motorerne blev hold fast med friktion.



Figur 24 - Billede af testopsætning af hjul og motorer



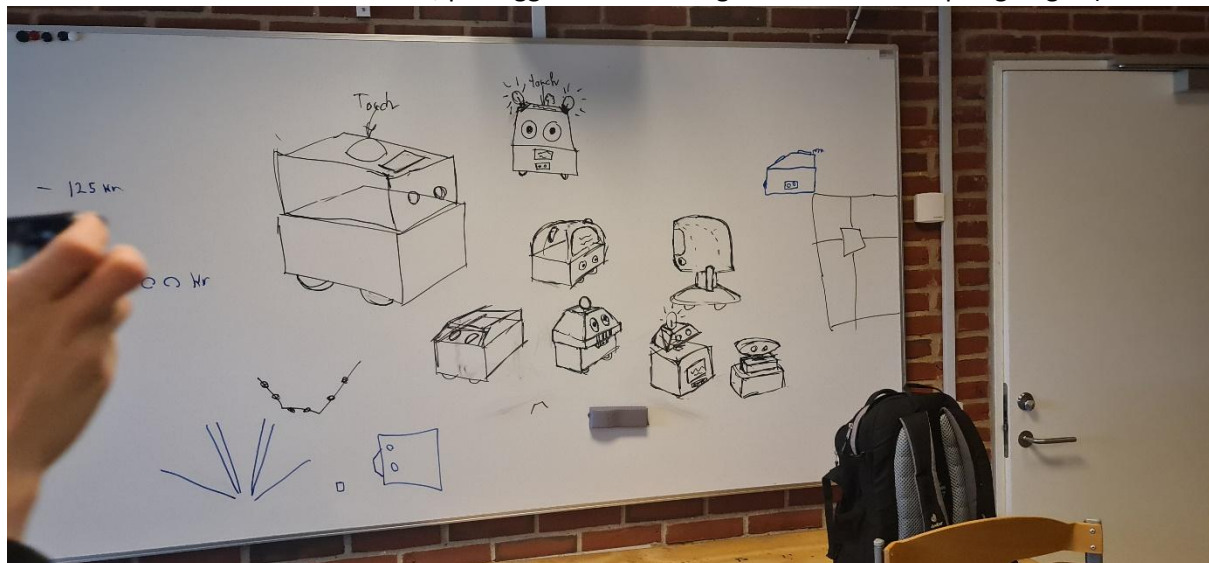
Figur 25 - Screenshot af 3D modellen til motor mount

Motorerne og hjulene kunne dermed sættes fast til en plade, hvorpå alt elektronikken kunne sammensættes. Der blev lavet huller ud fra motorerne, så ledninger kunne komme gennem til undersiden af robotten (Figur 26).

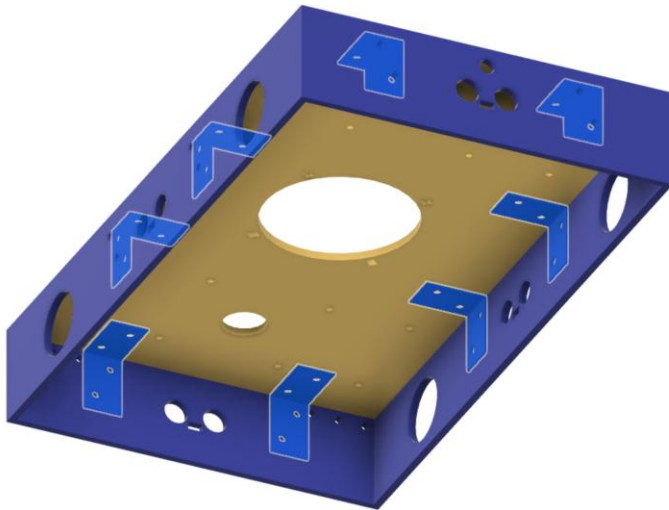


Figur 26 - Screenshot af bunden af 3D modellen af robotten

Herefter blev karrosseriet udformet, på baggrund af de tidligere nævnte koncepttegninger (

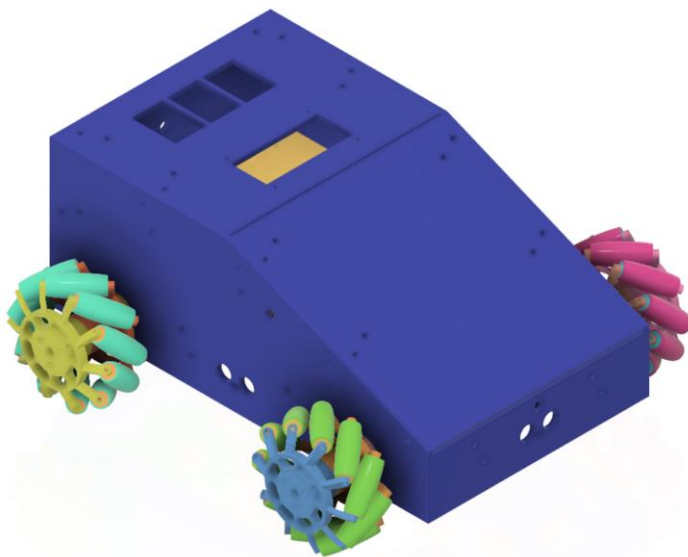


Figur 5). Først blev siderne af bunden på robotten lavet (Figur 27). De var med til at stabilisere motorerne, samt give plads til at montere ultrasoniske sensorer, samt lydsensorer. Bund siderne var fastmonteret med vinkelbeslag, som set på Figur 27.



Figur 27 - Screenshot af bund siderne, samt vinkelbeslag

Toppen af robotten, blev også fastmonteret med vinkelbeslag, samt blev der lavet huller til de resterende lydsensorer, skærmen og tryksensorerne (Figur 28). Faktisk var det på dette tidspunkt i processen, ikke tryksensorer, men selvlavet capacitive touch sensorer. Men pga. måden sådanne touch sensorer virker, var det ikke muligt at placere dem tæt på hinanden (Capacitive touch sensorerne og tryksensorerne beskrives yderligere i "Komponent opsætning -> Tryksensorer"). Derfor blev de senere udskiftet med tryksensorer. Denne ændring var dog ikke en væsentlig fejl og påvirkede ikke robottens karosseri negativt.



Figur 28 - Screenshot af toppen af 3D modellen til robotten

Før delene blev laserskrået i akryl, blev det først skåret i pap, så det var muligt at prototype delene og se hvordan de sad sammen (Figur 29). Her var det også muligt at sætte fingerpeg på de sidste detaljer. F.eks. skulle robotten have en hale? Det kunne vi teste med denne prototype og se hvordan det så ud. Vi endte med at droppe ideen med en hale, men det blev dog fastlagt, at de ultrasoniske sensorer skulle sidde i bund siderne, frem for top siderne. Samt blev vi også bekræftet i sammensætningen af delene, og kunne derfor fortsætte til at skære det designet i det endelige materiale.

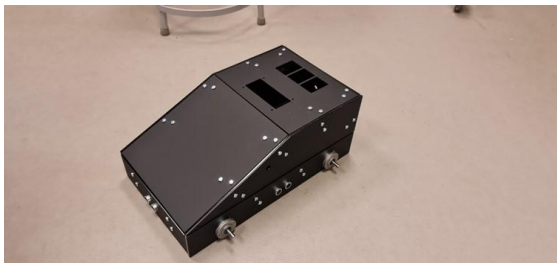


Figur 29 - Billede af pap-prototypen af robotten

Efter alle dele var laserskåret, var det bare at samle delene med 10mm M5 maskinskruer og møtrikker, ud fra de forud skåret huller. Se Figur 30, Figur 31, Figur 32 og Figur 33, for resultatet.



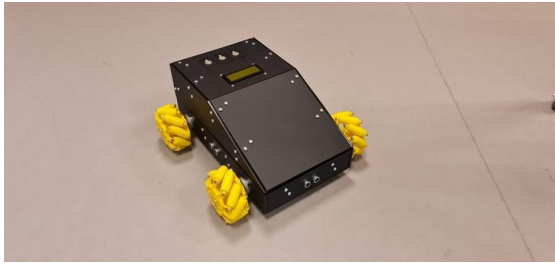
Figur 30 - Billede af den samlet bund af robotten



Figur 31 - Billede af den samlet bund og top af robotten



Figur 32 - Billede af den samlet robot med åben top



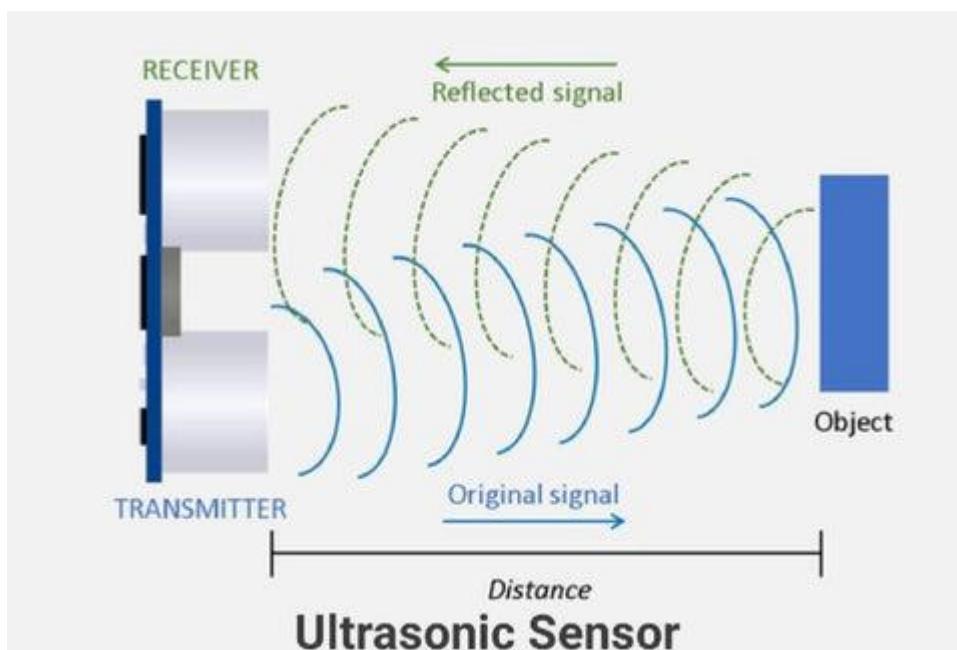
Figur 33 - Billede af den fuldt samlet robot

Komponent opsætning

Her beskrives, hvordan og, hvorfor komponenterne er opsat som de er. Samt gives der indblik i den relevante teori bag flere af komponenterne.

Ultrasoniske sensorer

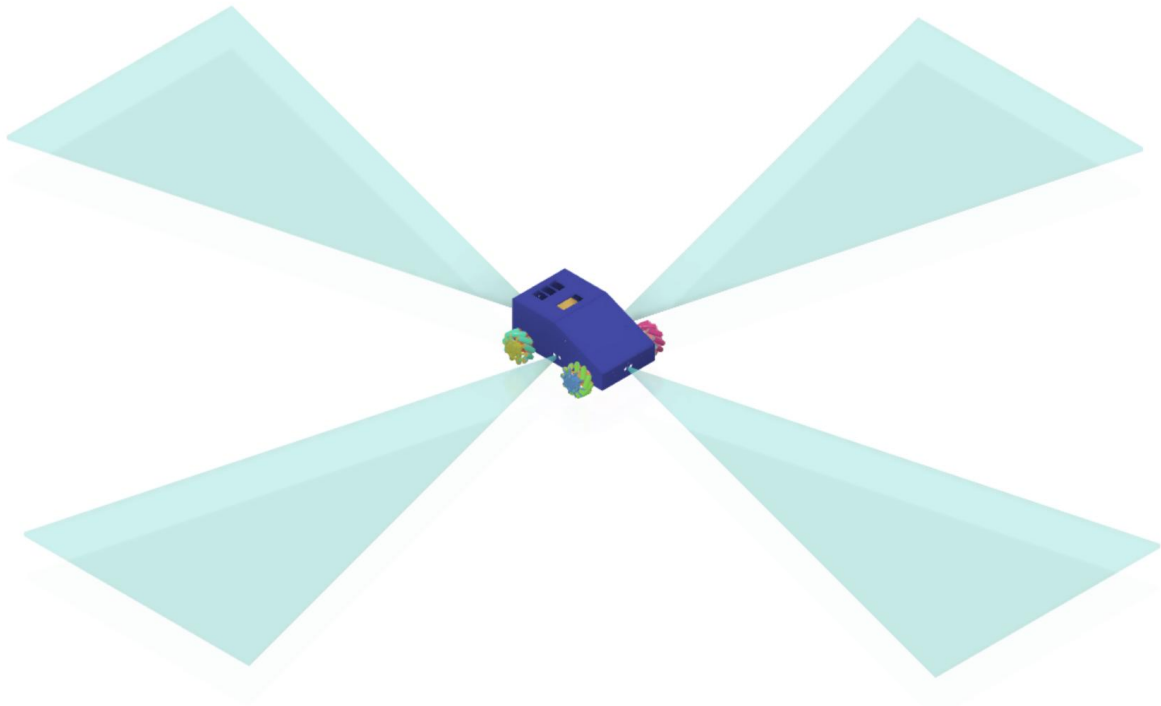
En ultrasonisk sensor, også kaldet en distance sensor, er et apparat der bruger ultrasoniske bølger, til at bestemme distance. De virker ved at sende en lydbølge ud fra dens afsender (Trigger), hvorefter der måles hvor lang tid der går før dens modtager (Echo), modtager lydbølgen tilbage. Hvis lydbølgen rammer et objekt, bliver lydbølgen reflekteret og sendt tilbage mod afsenderen, ud fra indfaldsvinklen. Dette kan ses på den nedenstående model (Figur 34). Mængden af tid det tager fra lydbølgen bliver sendt ud, til den kommer tilbage, kan derefter bruges til at udregne længden bølgen har rejst, ud fra lydens hastighed.



Figur 34 - Illustrering af hvordan ultrasoniske sensorer fungerer (robocraze, 2022)

Vi har valgt at placere de ultrasoniske sensorer tæt på jorden, i niveau med hjulene, for mest optimalt at kunne opfange forhindringer. Dog er der stadig store begrænsninger ved at bruge ultrasoniske sensorer til at opfange forhindringer. For det første, er de baseret på lydbølgers refleksion. Som tidligere nævnt, bliver lydbølgerne reflekteret ud fra indfaldsvinklen. Det betyder, at hvis indfaldsvinklen er for høj, bliver lydbølgen ikke reflekteret tilbage imod modtageren. Så sensorerne har svært ved at opfange buet, skrå eller enhver form, der leder lydbølgen væk fra modtageren. Derudover har de også et meget begrænset måle område. Ifølge HC-SR04's (den model vi benytter) datasheet, har sensoren en målevinkel på 15 grader, samt en max længde på 4 meter

(Electro Schematics, 2023). Dog har vi efter erfaring, begrænset max længden til 2 meter, da den er meget upræcis fra 2-4 meter. Denne meget smalle målevinkel skaber en del blinde vinkler. Dette kan visualiseres på Figur 35 (For reference, er robotten ca. 40x40 cm., eller omkring det samme som et A4-ark (Hvis ikke hjulene ikke tælles med)).

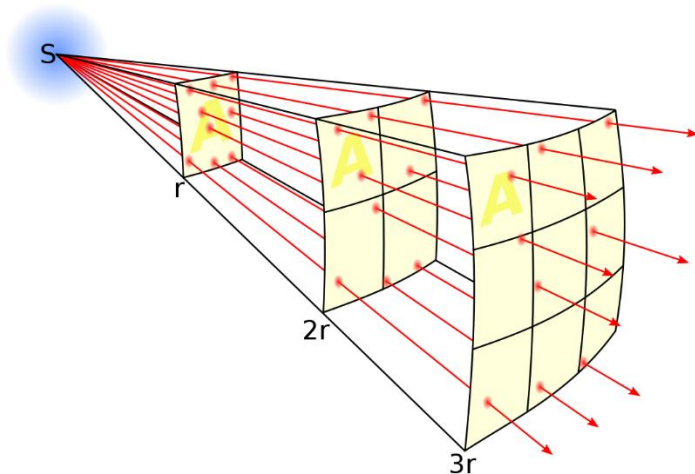


Figur 35 - Visualisering af robottens syn og blinde vinkler

Dette gør at robotten er meget god til at finde vægge eller andre store lige objekter, men ikke meget andet. En mulig løsning til problemet, er at tilføje flere sensorer, men eftersom stort set alle pins allerede var optaget på arduino megaen, samt realiteten, at der ikke var plads til det på robotten, var dette ikke en reel mulighed. I hvert fald ikke uden et re-design af robotten. Denne problematik allerede velkendt før produktionen begyndte, dog var det den bedste mulighed vi havde for at indføre kollisionsdetektion i robotten, med de færdigheder vi besidder.

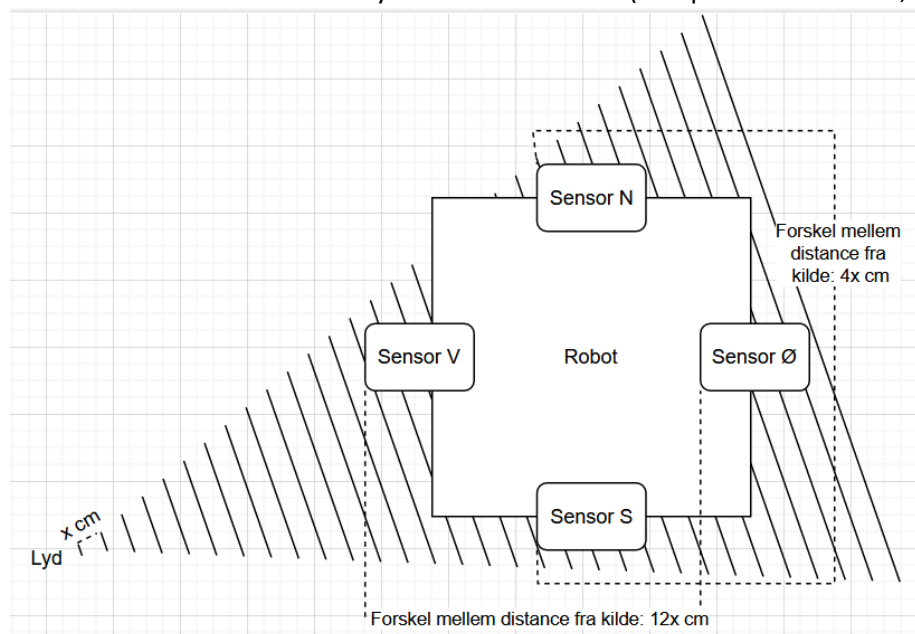
Lydsensorer

På samme måde som at der er opsat fire distancemålere, er der opsat fire lydsensor moduler på hver sin side af robotten. Lydsensorer er moduler, hvis digitale output reflekterer hvor høj en lyd mikrofonen på modulet opfanger. For at finde hvilken retning en lyd kommer fra, sammenlignes lydstyrken på de fire mikrofoner.



Figur 36 - Hvordan lydstyrke falder kvadratisk over distance. (Wikipedia Foundation, 2023)

Dette er muligt, da lyd er en bølge som spreder sig sfærisk. Det betyder at lydstyrken falder til $\frac{1}{r^2}$, som distancen, r , stiger, og at lydstyrken derfor falder signifikant over afstand, også over små mellemrum som dem mellem lydsensormodulerne. (Wikipedia Foundation, 2023)



Figur 37 - Opsætning af lydsensormoduler på robotten, samt eksempel på lyd der falder over distance.

Dog var der en høj usikkerhed på disse moduler, da det ene af komponenterne vi havde tilgængeligt, blev nødt til at have sin sensitivitet på det højest mulige, mens de andre moduler blev nødt til at have en lav sensitivitet for at matche den ikke-sensitve mikrofon.

Skærm

Skærmen der bruges på robotten, er et simpelt 4x20 LCD-display. Normalt skal denne skærm bruge 16 pins i en Arduino for at fungere, men ved hjælp af et I2C display interface, kan det reduceres ned til kun 4 pins. Dette er en vigtig ændring, da der er begrænset antal pins på en Arduino Mega.s

Højtaler

For at tilføje lyd til robotten, benyttede vi en standard 4 ohms højtaler sammen med en DF Player Mini. Dog var DF-playeren vores tredje løsning til højtaleren. Vi startede med at køre højtaleren med et SD kort modul, som læste lydfiler og afspillede dem på højtaleren. For at afspille lydfilerne på højtaleren, skulle de først konverteres til 8-bit mono lydfiler grundet den simplistiske natur af SD kort modulet og højtaleren.

Dog var dette for lavt til at kunne høres, især når motorerne også kørte. Vi blev nødt til at forstærke lyden. Derfor valgte vi at benytte en audio amplifier, til at forstærke lyden, stadig ved brug af SD kort modulet, til at afspille lyden fra. Den virkede, ved at vi førte 12 volt til amplifieren, i stedet for de tidligere 5 volt direkte til højtaleren.

Dette var dog stadig ikke højt nok, så vi skiftede til at bruge DF playeren i stedet. Det var vores første tanke, at vi skulle bruge både DF-playeren og amplifieren, men det viste sig, at DF-playeren var mere end høj nok af sig selv. Derudover gjorde DF-playeren det også muligt at håndtere alt lyd fra ét modul, eftersom den har en indbygget SD kort holder. Derfor kunne vi skille os af med både amplifieren og SD kort modulet.

Tryksensorer

De tre tryksensorer er opsat side om side, bagerst på toppen af robotten. De er med til at give robotten dens "ae" funktion. Det virker ved at der skal trykkes på en af de yderste tryksensorer, hvorefter der skal stryges hen over de andre tryksensorer i en blød bevægelse, ligesom når en hund aes.

Tidligere blev der benyttet selvlavet capacitive touch sensorer, men det blev ændret til tryksensorer, på baggrund af en grundlæggende fejl i vores design. Eftersom capacitive touch sensorer virker ved at måle ændringen i elektrisk ladning omkring det tilkoblet elektrisk ledende materiale, kunne de ikke virke i vores design. I designet var der under en centimeter mellem hver af sensorerne, hvilket var så tæt, at sensorerne kunne måle ændringen fra de andre sensorer. Derfor blev programmet forvirret, når der blev rørt ved enhver af sensorerne, da det blev vist som omtrenten ikke rørte dem, eller så blev de alle rørt.

Denne fejl skulle være fundet i vores indledende tests, men eftersom at vi testede sensorerne med et stort mellemrum, så opdaget vi ikke fejlen før vi allerede havde skåret det endelige design. Det gjorde dog ikke det store, eftersom vi havde beholdt de udskårne huller, så de kunne indsættes igen, sammen med tryksensorerne (Figur 38).



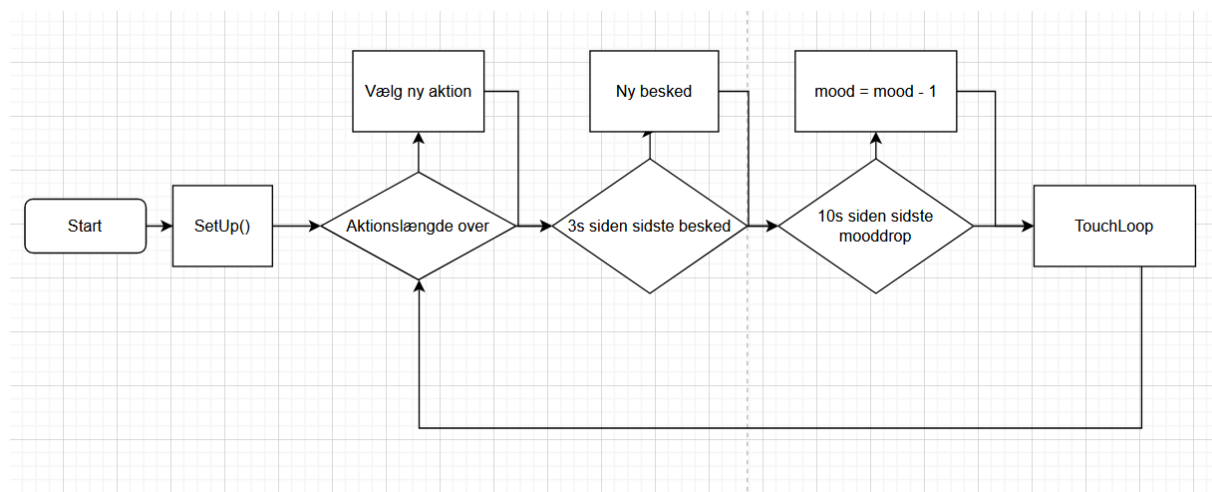
Figur 38 - Billede af tryksensorerne bag på robotten

Arduino kode

Allerede fra starten var det vigtigt at holde koden organiseret. Det ville f.eks. være meget uoverskueligt hvis alle de mange linjer kode var i samme script. Derfor blev det valgt at opdele koden i mange forskellige scripts. "QuadMotorControl" blev valgt til at være det script, der rent faktisk kørte når Arduinoen blev startet. Dette "QuadMotorControl" script starter med at importere kode

fra de andre scripts, som henholdsvis er i ".h" formatet. På den måde behandles alle de andre scripts som importeret libraries, hvorfra "QuadMotorControl" scriptet kan bruge variabler og funktioner. For at disse andre scripts kunne dele information med hinanden, blev der nødt til at blive skabt et script til at holde styr på variablerne. Dette script hedder "Variables.h", og er det første script som der importeres i "QuadMotorControl". "Variables.h" bliver nødt til at importeres først, fordi ellers kan de andre library scripts ikke dele information mellem hinanden.

Robottens kode bestod af flere dele. For at gøre det overskueligt at se, hvilken kode der hang sammen med hvad, er koden sat op sådan at filen "QuadMotorControl" har henholdsvis setup loopet samt main loopet, hvorfra alle andre funktioner bliver kaldt, hvis der er gået en hvis tidsmængde siden sidste gang den givne funktion blev kaldt. Dette gælder funktioner som omhandler robottens aktioner, ActionPicker(), tilfældige beskeder som repræsenterer robottens tanker, MsgPicker(), mood counteren, samt TouchLoopet, som tjekker berøring af det følesensitive område på robotten (**Fejl! Henvisningskilde ikke fundet.**).



Figur 39 - Flowchart for kodens overordnede funktion

Motor kontrol

Motorkontrollen af robotten, er opsat gennem tre scripts. Det første script er "MotorSetup.h" (Bilag 8). Her bliver alle motor pinsne sat op. Robotten er styret af fire motorer, som kører hjulene. De motorer er opdelt i forrest og bagerst, som er fordelt på de to motorkontrol units på robotten. Motor setups formål, er bare at sætte alle motor pins til output (Figur 40). Funktionen "MotorSetup" kan køres fra main loopet i "QuadMotorControl.ino" scriptet (Bilag 2), for at sætte alle pinsne op, uden at rode main scriptet.

```
void MotorSetup() {  
    pinMode(TLEN, OUTPUT);  
    pinMode(BLEN, OUTPUT);  
  
    pinMode(TREN, OUTPUT);  
    pinMode(BREN, OUTPUT);  
  
    pinMode(TLM_1, OUTPUT);  
    pinMode(TLM_2, OUTPUT);  
    pinMode(TRM_1, OUTPUT);  
    pinMode(TRM_2, OUTPUT);  
  
    pinMode(BLM_1, OUTPUT);  
    pinMode(BLM_2, OUTPUT);  
    pinMode(BRM_1, OUTPUT);  
    pinMode(BRM_2, OUTPUT);  
}
```

Figur 40 - Udsnit af MotorSetup funktionen fra MotorSetup.h

Efter motor setup, kommer "DirectionChange.h" (Bilag 5) og "DirectionEnum.h" (Bilag 6) som er de script der håndterer nem bevægelsen af robotten. DirectionEnum er script der opretter en enum med de forskellige retninger robotten kan bevæge sig (Figur 41). Derudover oprettes der også id-navne til hver af retningerne (Figur 42), så vi ikke skulle huske hvad hvert tal korresponderede med i enum'en, men bare kunne skrive "North", hvis den skulle køre frem.

```
enum Direction {  
    Stop,  
    North,  
    South,  
    East,  
    West,  
    NorthEast,  
    NorthWest,  
    SouthEast,  
    SouthWest,  
    RotateLeft,  
    RotateRight  
};
```

Figur 41 - Udsnit af Direction enum fra DirectionChange.h

```
#define IDNAME(name) #name
const char* directionNames[]
= {
    IDNAME(Stop),
    IDNAME(North),
    IDNAME(South),
    IDNAME(East),
    IDNAME(West),
    IDNAME(NorthEast),
    IDNAME(NorthWest),
    IDNAME(SouthEast),
    IDNAME(SouthWest),
    IDNAME(RotateLeft),
    IDNAME(RotateRight)
};
```

Figur 42 - Udsnit af IDNAME char array fra DirectionChange.h

Disse enum værdier bliver brugt i "DirectionChange.h" scriptet, som sørger for at robotten kan bevæge sig. Denne funktion virker ved at der sendes en retning, samt en hastighed, hvor scriptet så ændre retningen af de forskellige motorer, ift. hvordan robotten skal bevæge sig. F.eks. hvis robotten skal bevæges nord, sættes alle motorer til at køre frem. Der kan ses på Figur 43, hvordan "case North" sætter de forskellige motorers værdier. Motorernes pins er navngivet ved det første bogstav, der beslutter om det er et forhjul eller et baghjul. T for top og B for bottom. Derefter er der siden motoren er på. L for left og R for right. Til sidst er det et M for motor. Dette er så efterfulgt af inputtet på motoren. _1 er frem og _2 er tilbage. Så når frem (_1) sættes til HIGH, betyder det at den tændes. Derfor skal tilbage (_2) sættes til LOW, altså sluk. Til sidst er der EN, hvis der står EN i slutningen, er det til motorkontrol unitsne. EN er enable.

```
case North:
    digitalWrite(TLM_1, HIGH);
    digitalWrite(TLM_2, LOW);

    digitalWrite(TRM_1, HIGH);
    digitalWrite(TRM_2, LOW);

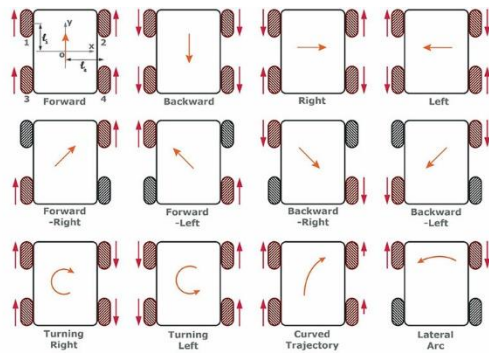
    digitalWrite(BLM_1, HIGH);
    digitalWrite(BLM_2, LOW);

    digitalWrite(BRM_1, HIGH);
    digitalWrite(BRM_2, LOW);

    analogWrite(TLEN, speedValue);
    analogWrite(TREN, speedValue);
    analogWrite(BLEN, speedValue);
    analogWrite(BREN, speedValue);
    break;
```

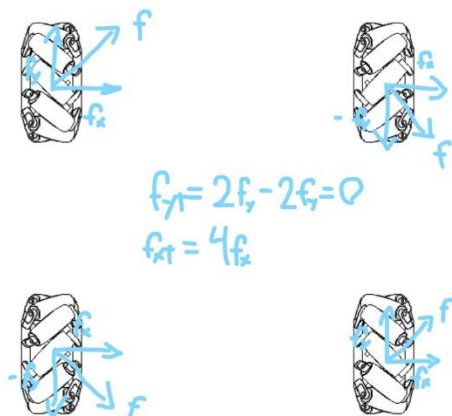
Figur 43 - Udsnit af North casen fra DirectionChange.h

For at få robotten til at køre i alle retninger, skal hjulene roteres meget specifikt, eftersom der gøres brug af mecanum hjul. Grundet fysikken bag hjulenes kræfter, kan den køre i alle retninger, hvis hjulene roteres på en specifik måde (Figur 44)



Figur 44 - Illustrering af retningsinstruktioner til hjul (Jafari, 2023)

Måden det virker, er udligning af kræfter. Hvis der kigges på kræfterne der er i hulet, når det roteres, giver det god mening. Sig der skal køres mod højre. Hvis alle kræfterne af hjulene tegnes (Figur 45), kan net kræften i x og y retningerne udregnes. Tag specifikt et kig på det øverste forreste hjul. Hvis der kigges tilbage på den tidligere Figur 44, skal det hjul dreje bagud (ned af). På grund af den 45 grader rotation af rollerne, vil hjulet blive presset op mod højre (Nord Øst). Herefter kan alle kræfterne udregnes. Alle kræfterne på y-aksen, bliver udlignet fra hjulet over eller under. Og alle hjul bliver presset mod højre, hvilket får hele robotten til at køre til højre. Det er derfor det er så vigtigt, hvordan hjulene er placeret. Hvis der lægges mærke til rollerne, kan det ses at de alle peger mod robotens centrum, som et kryds. Det er denne orientering der gør at hjulene er holonomiske, og det er sådan hjulene bliver kontrolleret i Direction Change scriptet.



Figur 45 - Skitsering af alle kræfterne der påvirker robotens bevægelse mod højre

Avoidance system

Formålet med "AvoidanceSystem.h" scriptet (Bilag 4), er at sikre robotten ikke kører ind i obstruktioner. Systemet er opbygget på et "blocking loop". Når først avoidance systemet er blevet aktiveret, hvilket sker når robotten kommer for tæt på objekter, bliver der startet et loop, som først stopper når robotten igen er en sikker distance for objekterne omkring den. Ud fra hvilke retninger robotten er blokeret fra, vælger robotten den mest optimale retning i den modsatte retning. De retninger er hardcoded ift. hvilke sensorer der opfanger objekter tæt på sig. Avoidance funktionen bliver kørt i main loopet. Når en af de fire ultrasoniske sensorer måler en distance der er under 10 centimeter, bliver et rekursivt loop startet, som først stopper når alle ultrasoniske sensorer måler en distance der er over eller lig med 10 centimeter. På Figur 46, kan der ses det store if-else statement, der vælger hvilken retning robotten skal gå. F.eks. hvis der blokeres fra øst, bliver der spurgt "distance_E < minDistance", herefter bliver der brugt den tidligere nævnte ChangeDirection

funktion, til at sætte retningen til vest "ChangeDirection(West, 255);" og så bliver der printet hvilken vej robotten går imod, samt antallet af blokeringer "Serial.println("West - 1");".

```
void Avoidance(float minDistance){
    Serial.print("Avoiding - ");
    SensorLoop();

    if (distance_N < minDistance && distance_E < minDistance &&
        distance_S < minDistance && distance_W < minDistance){
        ChangeDirection(RotateLeft, 255);
        Serial.println("RotateLeft - 4");
    }
    else if (distance_E < minDistance){
        ChangeDirection(West, 255);
        Serial.println("West - 1");
    }

    if (distance_N >= minDistance && distance_E >= minDistance &&
        distance_S >= minDistance && distance_W >= minDistance){
        ChangeDirection(Stop);
        avoiding = false;
        return;
    }

    Avoidance(minDistance);
}
```

Figur 46 - Udsnit af dele af avoidance koden fra AvoidanceSystem.h

Dog er den vigtigste linje, der får systemet til at virke, linjen "SensorLoop();" (Figur 46, specifikt tredje linje). Denne linje er med til at sikre, at distance målingerne også opdateres. Eftersom at det er et blocking loop, bliver sensorerne ikke opdateret uden denne linje. Funktion tilhører scriptet "SensorControl.h" (Bilag 10) og det er her distancerne måles. Scriptet har tre funktioner, en SensorSetup funktion til at sætte pinsne op, ligesom ved motorerne. På Figur 47, er det muligt at se hvordan sensorerne er sat op. Der er fire sensorer i alt, hver med deres egen echo og trigger pin. Trigger pinen er den pin, der sender lydbølgen ud, når den aktiveres. Derfor er den sat til OUTPUT. Echo pinen er den pin, der modtager lydbølgen, når den er reflekteret tilbage.

```
void SensorSetup(){
    pinMode(TRN, OUTPUT);
    pinMode(TRE, OUTPUT);
    pinMode(TRS, OUTPUT);
    pinMode(TRW, OUTPUT);
    pinMode(ECN, INPUT);
    pinMode(ECE, INPUT);
    pinMode(ECS, INPUT);
    pinMode(ECW, INPUT);
}
```

Figur 47 - Udsnit af SensorSetup funktionen fra SensorControl.h

Herefter er der funktionen SensorLoop, som er med til at opdatere de nuværende distanceværdier, opdatere en liste med de seneste værdier og printe værdierne. Dog printes værdierne kun, hvis en bool (showSensorInfo) er true (Figur 48).

```
void SensorLoop() {  
    distance_N = GetDistance(TRN, ECN);  
    distance_E = GetDistance(TRE, ECE);  
    distance_S = GetDistance(TRS, ECS);  
    distance_W = GetDistance(TRW, ECW);  
  
    if (millis() - distTimer > 30) {  
        distTimer = millis();  
        distRawN[distCount] = distance_N;  
        distRawS[distCount] = distance_S;  
        distRawE[distCount] = distance_E;  
        distRawW[distCount] = distance_W;  
  
        distCount++;  
        if (distCount == 20) {distCount = 0;}  
    }  
  
    if (showSensorInfo) {  
        Serial.print("N: ");  
        Serial.print(distance_N);  
        Serial.print(" - E: ");  
        Serial.print(distance_E);  
        Serial.print(" - S:");  
        Serial.print(distance_S);  
        Serial.print(" - W:");  
        Serial.println(distance_W);  
    }  
}
```

Figur 48 - Udsnit af SensorLoop funktionen fra SensorControl.h

distance_N (Nord), E (Øst), S (Syd) og W (Vest) opdateres via. GetDistance funktionen. Funktionen tager to pins (ints) som parametre, for at kunne genbruge samme funktion til hver sensor. GetDistance funktionen returner en float værdi, som er distancen sensoren opfanger. Funktionen virker ved at starte med at sætte trigger pinen til LOW, som en sikkerhedsforanstaltning, hvis værdien skulle være sat til HIGH et andet sted. Herefter bliver der ventet 2 mikrosekunder, for at sikre at pinens spænding faktisk er faldet. Så bliver lydbølgen sendt ud, ved at tænde trigger pinen, ved at sætte den til HIGH i 10 mikrosekunder, før den sættes til LOW igen. Herefter benyttes pulseIn funktionen, til at måle tiden det tog før Echo pinen modtager lydbølgen igen. Eftersom at Echo pinen er et input, sender den et signal, når den modtager lydbølgen. Denne tidsværdi kan så bruges til at udregne distancen lydbølgen havde rejst. Vi ved at lydbølgen bevægede sig med lydens hastighed, og vi ved at lydens hastighed er $343 \frac{m}{s}$. Derudover ved vi at pulseIn funktionen returner en værdi i millisekunder, hvilket svarer til $\frac{1}{1000}$ af et sekund. Til sidst ved vi at pulsen er 10 mikrosekunder lang. Derfor kan vi udregne hvad vi skal gange "duration" værdien med, for at få resultatet i centimeter.

Eftersom at $\frac{343 \frac{m}{s}}{\frac{1000 ms}{10 \mu s}} = 0.0343 \text{ cs}$ (Centi – sekunder). Derfor bliver resultatet i centimeter. Dog er resultatet stadig det dobbelte af det endelige resultat, eftersom lydbølgen både skal bevæge sig hen til objektet, men også reflekteres tilbage. Derfor divideres resultatet med 2. Til sidst bliver resultatet clamped til en min værdi på 5 centimeter og en max værdi på 2 meter, hvorefter resultatet returneres (Figur 49).

```
float GetDistance(int TriggerPin, int EchoPin){
    digitalWrite(TriggerPin, LOW);
    delayMicroseconds(2);
    digitalWrite(TriggerPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(TriggerPin, LOW);
    float duration = pulseIn(EchoPin, HIGH);
    float distance = (duration * 0.0343) / 2;
    distance = constrain(distance, minDistance, maxDistance);
    return distance;
}
```

Figur 49 - Udsnit af GetDistance funktionen fra SensorControl.h

Sound control

“SoundControl.h” scriptet (Bilag 11), er det script, der håndterer alt lyden. Det er her DF player mini - en bliver sat op i “SoundPlayerSetup” funktionen, som set på Figur 50.

```
SoftwareSerial softwareSerial(10, 11);

DFRobotDFPlayerMini player;

void SoundPlayerSetup() {
    Serial.begin(9600);
    softwareSerial.begin(9600);

    if (player.begin/softwareSerial)) {
        player.volume(volumeLevel);
        player.play(1);
    } else {
        Serial.println("Connecting to DFPlayer Mini failed!");
    }
}
```

Figur 50 - Udsnit af SoundPlayerSetup funktionen fra SoundControl.h

“SoundControl.h” scriptet (Se bilag x), er det script, der håndterer alt lyden. Det er her DF player mini -en bliver sat op i “SoundPlayerSetup” funktionen, som set på figur x.

```
SoftwareSerial softwareSerial(10, 11);
DFRobotDFPlayerMini player;

void SoundPlayerSetup() {
    softwareSerial.begin(9600);

    if (player.begin/softwareSerial)) {
        player.volume(volumeLevel);
        player.play(1);
    } else {
        Serial.println("Connecting to DFPlayer Mini failed!");
    }
}
```

Her bliver der først oprettet en software serial, som er med til at kommunikere med SD kortet på modulet. Her bliver der gjort brug af pin 10 og 11, som er hhv. RX (Receiver) og TX (Transmitter) - pins. Derudover bliver der oprettet en DFPlayerMini, som del af “DFRobotDFPlayerMini” library’et,

der tager en software serial som parametre. Denne software serial initieres ved "softwareSerial.begin(9600);" (Her er 9600 den baud rate som arduinoen kommunikerer på). Herefter tjekkes der om playerens software serial er begyndt. Hvis ja, indstilles lydniveauet, samt afspilles en startup lyd, for at indikere at robotten er startet (Figur 50).

Herefter bliver funktionerne "PlaySound" og "PlayRandomSound" benyttet til enten at afspille en specifik lyd, eller afspille en tilfældig lyd ud fra en liste af lyde. Grunden til at vi har PlayRandomSound funktionen, er fordi vi til nogle humører, har flere forskellige lyde der kan afspilles, for at give noget variation. Derfor har vi oprettet lister af de lyde, så vi kan afspille én tilfældig lyd. Måden afspilningen virker, er ved at bruge funktionen "player.play(trackNr)" her er trackNr det index som lyden har på SD kortet. Det var desværre ikke muligt at afspille lydene ud fra lydfiles navn, så derfor har vi opstillet lydene i en enum, ligesom ved DirectionChange. Her korresponderer int'en fra enumen, det index lyden har på SD kortet. Herved kan vi afspille lydene ud fra navnet på nummeret (Figur 51).

```
enum SoundTrack {  
    Nysgerrig1,  
    Passiv1,  
    Passiv2,  
    Passiv3,  
    Snork1,  
    Sur1,  
    Sur2,  
    Sur3,  
    Sang1  
};  
  
#define IDNAME(name) #name  
const char* soundTrackNames[] = {  
    IDNAME(Nysgerrig1),  
    IDNAME(Passiv1),  
    IDNAME(Passiv2),  
    IDNAME(Passiv3),  
    IDNAME(Snork1),  
    IDNAME(Sur1),  
    IDNAME(Sur2),  
    IDNAME(Sur3),  
    IDNAME(Sang1)  
};
```

Figur 51 - Udsnit af SoundTrack enum, samt IDNAME char array fra SoundControl.h

```
enum SoundTrack {  
    Nysgerrig1,  
    Passiv1,  
    Passiv2,  
    Passiv3,  
    Snork1,  
    Sur1,  
    Sur2,  
    Sur3,  
    Sang1  
};  
  
#define IDNAME(name) #name  
const char* soundTrackNames[] = {  
    IDNAME(Nysgerrig1),  
    IDNAME(Passiv1),  
    IDNAME(Passiv2),  
    IDNAME(Passiv3),  
    IDNAME(Snork1),  
    IDNAME(Sur1),  
    IDNAME(Sur2),  
    IDNAME(Sur3),  
    IDNAME(Sang1)  
};
```

Selve funktionerne er bygget op meget ens. Først bliver der tjekket om højttaleren allerede er i gang med at afspille en lyd, hvis ikke, afspiller den en ny lyd ud fra det parametre (SoundTrack) der er sendt med funktionen (Figur 52).

```
void PlaySound(SoundTrack track) {  
    if (digitalRead(7) == 1) {  
        player.play(track + 1);  
    }  
}  
  
void PlayRandomSound(SoundTrack tracks[]) {  
    if (digitalRead(7) == 1) {  
        int rnd = random(0, sizeof(tracks) / sizeof(int));  
        player.play(tracks[rnd] + 1);  
    }  
}
```

Figur 52 - Udsnit af PlaySound og PlayRandomSound funktionerne fra SoundControl.h

```
void PlaySound(SoundTrack track){  
    if (digitalRead(7) == 1){  
        player.play(track + 1);  
    }  
}  
  
void PlayRandomSound(SoundTrack tracks[]){  
    if (digitalRead(7) == 1){  
        int rnd = random(0, sizeof(tracks) / sizeof(int));  
        player.play(tracks[rnd] + 1);  
    }  
}
```

PlayRandomSound funktionen, virker på samme måde, men der er et trin mere i processen, nemlig at vælge lyden. Lyden bliver valgt ud fra det array af lyde der sendes med funktionen som parametre. Her bliver der valgt et tilfældigt tal mellem 0 og længden af arrayet, som derefter bliver afspillet. Arduino kode har ikke en indbygget funktion til at få længden af et array. Derfor bliver vi nødt til at dividere bit-længden af hele arrayet, med bit-længden af den type arrayet er. Dermed kan vi finde ud af hvor mange ints arrayet indeholder, ergo længden af int arrayet (Figur 52).

Ved begge funktioner bliver der lagt én til det endelige trackNr, for at få fat i den korrekte lyd.

Touch sensor

TouchSensor.h scriptet tjekker om robotten er blevet "aget på hovedet" (Bilag 13). Robotten har tre tryksensorer på toppen, som hver især kan registrere om de bliver rørt ved. TouchSensor.h scriptet tager imod input fra alle tryksensorerne og tjekker om de er blevet rørt i den rigtige sekvens. Den rigtige sekvens vil enten være højre tryksensor, midterste tryksensor og til sidst venstre tryksensor, eller den omvendte rækkefølge. De to rigtige sekvenser er til for at sikre at brugeren nogenlunde skal efterligne at age robotten. Scriptets TouchLoop funktion består af en masse if statements som sikrer at tryksensorerne er blevet rørt i den korrekte rækkefølge. Der er samtidigt CheckTime funktionen som sikrer at der ikke er gået mere end et sekund siden den forrige knap i rækkefølgen er blevet rørt, dette gøres for at sikre den age lignende bevægelse finder sted.

TouchSensor.h scriptet tjekker om robotten er blevet "aget på hovedet" (bilag x). Robotten har tre tryksensorer på toppen, som hver især kan registrere om de bliver rørt ved. TouchSensor.h scriptet tager imod input fra alle tryksensorerne og tjekker om de er blevet rørt i den rigtige sekvens. Den rigtige sekvens vil enten være højre tryksensor, midterste tryksensor og til sidst venstre tryksensor, eller den omvendte rækkefølge. De to rigtige sekvenser er til for at sikre at brugeren nogenlunde skal efterligne at age robotten. Scriptets TouchLoop funktion består af en masse if statements som sikrer at tryksensorerne er blevet rørt i den korrekte rækkefølge. Der er samtidigt CheckTime funktionen som sikrer at der ikke er gået mere end et sekund siden den forrige knap i rækkefølgen er blevet rørt, dette gøres for at sikre den age lignende bevægelse finder sted. Scriptets flow kan ses i dets tilhørende flowdiagram (bilag x).

Action chooser

AktionChooser er det script, som styrer hvilke aktioner robotten kan tage og hvornår. Ved brug af de allerede definerede funktioner i de scripts, som er ansvarlige for de forskellige sensorer, defineres der her en aktionslængde (ActionTime) samt evt. en besked eller andet nødvendig kode for aktionerne. Efterfølgende vælges der, an på værdien af "mood", en tilfældig aktion ud fra vægtningen af de individuelle aktioner, som beskrevet i kapitlet om interaktioner.

Dette er gjort med fem if/else statements med humørets værdi som variabel, hvor der efterfølgende bliver taget et tilfældigt tal mellem summen af aktioneres vægte og nul, hvorefter endnu en række if/else statements definerer hvilken aktion der vælges.

Aktionerne robotten kan vælge er efterforsk, sang, lur, snurreRundt, opmaerksomhed, selvhad, vrede, samt nysgerrig, hvis dependencies kan ses i tabellen forned.

Aktion	Dependencies
Efterforsk	MotionDirection
Sang	-
Lur	FindWall(true)
snurreRundt	-
Opmaerksomhed	MotionDirection
Selvhad	FindWall(false)
Vrede	MotionDirection
Nysgerrig	DetectSound(2000, 128)

Screen

Screen scripted kontrollerer hvilke beskeder robotten viser på hvilke tidspunkter. Der er oprettet en string array til hvert af de fem humør med mellem seks og otte beskeder i hver. I dette script er funktionen MsgPicker også, som vælger tilfældige beskeder an på humøret når kaldt. Dette gøres ved at tage et tilfældigt tal mellem nul og listelængden, og derefter skrive beskeden på dette indeks af den pågældende liste.

Distance measure

I dette script måles det, hvilken distance der er fra robottens fire ultrasoniske distancemålere til det nærmeste objekt, samt om dette objekt er i bevægelse. Ultrasoniske distancemålere virker ved, at de sender pulse af ultrasonisk lyd, som derved bliver reflekteret af genstande den rammer. Efterfølgende måler den hvor mange millisekunder det tager, før bølgen når tilbage til sensoren.

Koden består af tre overordnede dele. De to funktioner som henholdsvis finder hvilken retning der er en bevægelse samt funktionen som finder det nærmeste objekt. Den sidste del er den funktion der bruges til at filtrere den indsamlede data, for at få et mere akkurat resultat.

Filteret opretter først et array ud fra den originale data. Efterfølgende sorteres denne data i stigende rækkefølge, og til sidst tages gennemsnittet af den midterste halvdel af dataarraysne (Figur 53).

```
float filter(float filterArray[], float rawArray[], bool comp) {  
    // For første del af arrayen.  
    if (!comp) {  
        for (int i = 0; i < 10; i++){  
            filterArray[i] = rawArray[i];  
        }  
    }  
    // F  
    else {  
        for (int i = 10; i < 20; i++){  
            filterArray[i - 10] = rawArray[i];  
        }  
    }  
  
    // Sorter i data i stigende rækkefølge.  
    for (int i = 0; i < 9; i++) {  
        for (int j = i + 1; j < 10; j++) {  
            if (filterArray[i] > filterArray[j]) {  
                float swap = filterArray[i];  
                filterArray[i] = filterArray[j];  
                filterArray[j] = swap;  
            }  
        }  
    }  
  
    // Find gennemsnittet af de midterste data.  
    double sum = 0;  
    for (int i = 3; i < 7; i++) {  
        sum += filterArray[i];  
    }  
  
    return sum / 4;  
}
```

Figur 53 - Udsnit af kode, som filtrerer dataen for distancerne.

Der er her blevet valgt kun at bruge de ti nyeste datapunkter for arraysne. Dette er for at minimere mængden af tid det tager at finde distancen, da hver måling har en cooldown, samt for at minimere den brugte hukommelse i arduinoen. Algoritmen der er blevet brugt til at sortere dataerne med er en bubble sort algoritme, altså essentielt et brute force, hvor data bliver byttet om hvis et stykke data er mindre end det næste, hvilket har en performance på $O(n^2)$. Dette er ikke optimalt, men grundet det lille datasæt ville en optimeret sortering ikke have gjort en mærkbar forskel.

MotionDirection() fungerer ved, at den opretter en variabel for distancen fra hver af de fire retninger, samt en variabel som er gennemsnittet af de efterfølgende målinger, som hver tages gennem filter funktionen. Efterfølgende tages den absolutte værdi af distancen for retning som bliver trukket fra med dens sammenligningsdata. Hvis denne værdi er højere end fem, ændres retningen af robotens kørsel til retningen bevægelsen kommer fra. Grunden til at tallet fem er valgt, er for at give en margin for, hvornår en bevægelse skal tælles med, da der kan være en naturlig fluktuerende mellem tal fra sensoren selv.

Den sidste funktion, FindWall, virker ligeledes ved at filtrere data fra de forskellige retninger. Efterfølgende tjekker funktionen om hver af retningens værdi er mindre end de andre retningers værdi. Den mindste værdi sættes derefter som den retning robotten kører. Herefter, hvis robotten

ikke skal sove, kører robotten ind i objektet tre gange, ved at køre frem og tilbage ind i objektet, når den er inden for en vis distance af objektet.

Sound detection

Lignende distancemåleren, oprettes og filtreres fire arrays, hvor et gennemsnit af de midterste data tages. Herefter sammenlignes nord med syd, og øst med vest, for at bestemme hvilken retning lyden kommer fra. Efterfølgende sammenlignes de to valgte retninger, og hvis en er markant højere end den anden, sættes den kørende retning til denne. Hvis ikke, vælges den skrå retning mellem dem.

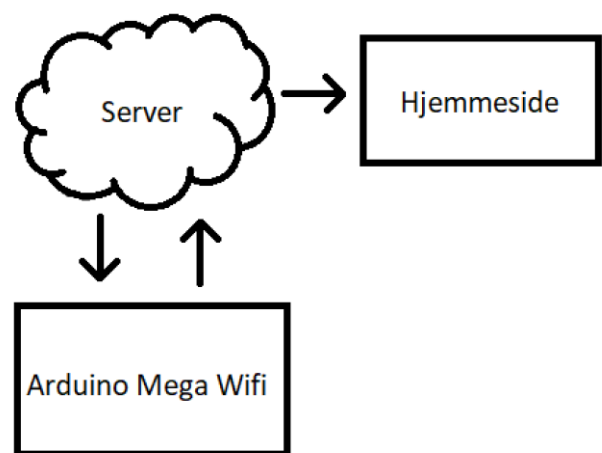
Arduino Mega WiFi skandalen

Under planlægningen af robotten, blev det besluttet at opbevare robottens adfærdsvariabler på en server. Der skete dog en serie af fejl, som forhindrede dette i at ske.

Planen

Ifølge interaktionsdesignet skulle robotten have forskellig adfærd baseret på variabler, der dikteret robottens humør. Det var besluttet at opbevare robottens humør-variabler på en server, separat fra selve Arduinoen. Samtidig skulle et program køre på serveren, i modsætning til på Arduinoen, med formålet at udregne robottens nuværende humør. Dette var en afgørende beslutning for at opnå to formål.

Først og fremmest ville det resultere i et mere stabilt adfærdssystem. Selv hvis robotten mistede strøm, kunne robotten blot hente dens adfærdsvariabler fra serveren, når den blev tændt igen. Det betød altså, at gruppen ikke behøvede at skulle bekymre sig om tab af data eller ustabilitet i robottens adfærd under strømsvigt.



Figur 54 - Diagram over opbevaring og fremvisning af adfærdsvariabler ifølge den originale plan

For det andet ville de opbevarede variabler kunne logges og vises på en hjemmeside. Dette ville give en mulighed for at holde styr på robotten under brugertesten, selv når gruppen ikke ville være til stede.

For at kommunikere med en sådan server, er det nødvendigt at have et Wi-Fi modul. Dette Wi-Fi modul ville enten kunne være et separat modul fra selve Arduinoen, eller der kunne tages brug af en Arduino Mega WiFi. Fordelen, og grunden til at gruppen valgte denne løsning, er at en Arduino Mega WiFi har Wi-Fi modulet allerede indbygget. Vi, som gruppe, var allerede i planlægningsfasen bevidste om at det ville blive presset med antal ledige pins. Derfor valgte vi at bruge Arduino Mega WiFi'en, da den har lige så mange pins som en almindelig Arduino Mega, men derudover har den også et Wi-Fi modul indbygget, specifikt en ESP8266.

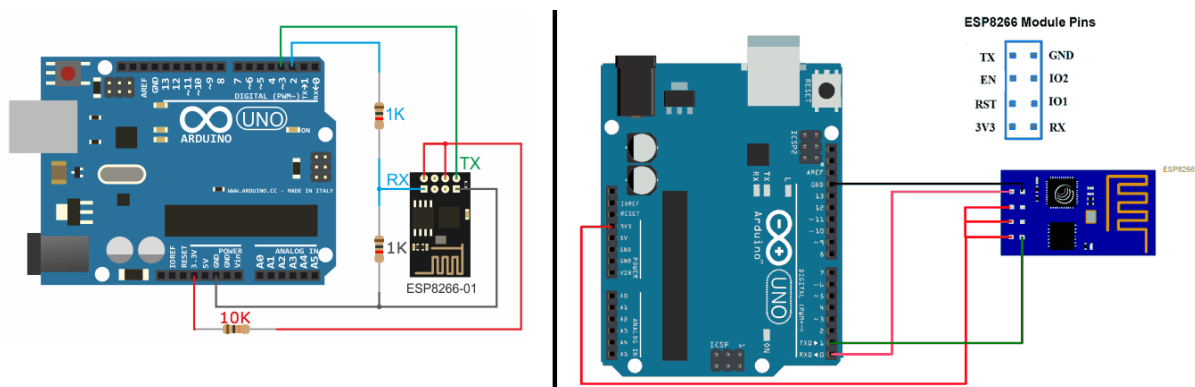
Serveren

Det var planlagt, at serveren ville være en computer, der sad kørende på skolen. Hvis det var en f.eks. bærbar computer på skolen, ville både den bærbare computer og Arduino Mega WiFi'en kunne forbinde til samme netværk, og på den måde dele information imellem hinanden. Denne computer ville køre et program skrevet i C#, som ville være ansvarligt for at beregne og opbevare adfærdsvariablerne. Disse adfærdsvariabler ville blive uploadet fra serveren, til et JSON dokument på et GitHub repository. Dette GitHub repository ville også indeholde koden til hjemmesiden, hvorpå

information såsom adfærdsvariabler, ville blive vist. GitHub har en service ved navn GitHub Pages (GitHub, 2023). Med GitHub Pages er det muligt at hoste en hjemmeside, ved blot at have noget HTML/CSS/JavaScript kode og et Node.js miljø sat op inde på et repository. Hjemmesiden ville bruge JavaScript kode til at læse JSON filen, og bruge de ekstraheret variabler til at logge og fremvise robotens adfærd.

Hvad gik galt?

Skolen havde ikke en Arduino Mega WiFi til rådighed, så sådan en Arduino blev nødt til at blive bestilt, med omkring en måned ventetid. Imens vi ventede på ankomsten af vores Arduino Mega WiFi, lavede vi nogle tests på en normal Arduino Mega med et ESP8266 modul. For at gøre dette blev vi nødt til at tilslutte ESP8266 modulet til den normale Arduino Mega. Vi brugte internettet til at finde ud af, hvordan opsætningen af ledningerne og eventuelle modstande skulle være, men der var mange forskellige svar.



Figur 55 - Eksempler af opsætning af ledninger fra Arduino Uno til ESP8266. Billeder hentet fra (Jeff-Paredes, 2017) og (ElectronicWings, 2023)

På grund af dette var det svært at finde ud af hvilken opsætning, der ville være den bedste. Derudover var alle eksemplerne lavet med en Arduino Uno og ikke en Arduino Mega. Når fejl opstod under opsætningen, var det derfor svært at finde ud af, om fejlen var skyld af koden eller opsætningen af ledningerne. Desuden krævede de forskellige opsætninger af ledninger forskellig kode. Dette gjorde det næsten umuligt at finde og eliminere fejl. Fremskridt på dette stadie var meget langsomt, og det blev derfor besluttet at fokusere på andre komponenter indtil Arduino Mega WiFi'en ankom. Efter omkring en måned ankom Arduino Mega WiFi modulet, men selvom ledning opsætningen ikke længere var et problem at være bekymret over, var fremskridt med det nye modul stadig langsomt. Adskillige moduler blev lagt i at forsøge blot at forbinde Arduino Mega WiFi modulet til internettet. Pludselig stoppede Arduino Mega WiFi'en med at tilslutte til alle computerne, så der blev det besluttet at opbevare adfærdsvariablerne på en almindelig Arduino Mega og at bruge tiden et andet sted.

Lyddesign

For at skabe en større følelse af interaktion mellem brugeren og robotten, valgte vi i gruppen at arbejde med lyd som en del af interaktionsdesignet. De primære mål med lyddesignet er at skabe feedback på interaktionerne som brugeren har med SENNEP, tiltrække opmærksomhed når den er i et rum, og give den en unik stemme, der bidrager til dens nuttede, kæledyrsagtige udtryk.

Én af de metoder, der bruges til at skabe feedback gennem lyd, er at robotens passive lyde ændrer sig alt efter dens humørværdi (se de tre lydfiler nedenfor). Dette giver forhåbentlig brugere et indtryk af robotens humør, selvom de ikke nødvendigvis aktivt interagerer med den.



1 - passiv.mp3



5 - passiv glad.mp3



7 - passiv trist.mp3

Robotten har også en række lyde, der afspilles som reaktion på bestemte interaktioner, som f.eks. hvis brugeren nusser den, eller hvis den udfører én af sine tilfældige handlinger som f.eks. at snurre rundt eller køre ind i en væg. Det er med til yderligere at koble lydsiden til resten af interaktionsdesignet, give robotten personlighed, og giver desuden brugeren direkte feedback igennem interaktionen.

Lyddesignet er udformet i FL Studio. Robottens "stemme" er designet i softwaresynthesizeren Omnisphere, og består af en synth-tone der stiger i tonehøjde når den spilles i længere tid. Derfra kan robottens lydeffekter laves med den samme synthesizer ud fra en kombination af lange toner og korte bip, der er med til at give et lydudtryk, der minder om menneskelignende robotter fra science fiction-genren - én af de store inspirationer for robottens stemme er R2D2 fra Star Wars (photoelectric, 2009). Det eneste af de korte stemmeklip, der ikke stammer fra den samme synth, er lydeffekten hvor robotten spinder efter at blive nusset på hovedet - det er produceret ud fra et klip af en kat der spinder¹, sendt igennem en række effekter som f.eks. digital forvrængning og bitcrusher, der bedre matcher robottens udtryk. Sangen, som S.E.N.N.E.P. synger når den er glad, er produceret med virtuelle instrumenter i FL Studio og derefter ligeledes processeret på denne måde.

For at tilføje lyd til robotten, benyttede vi en standard 4 ohms højttaler sammen med en DF Player Mini. Dog var DF-playeren vores tredje løsning til højttaleren. Vi startede med at køre højttaleren med et SD kort modul, som læste lydfiler og afspillede dem på højttaleren. For at afspille lydfilerne på højttaleren, skulle de først konverteres til 8-bit mono lydfiler grundet den simplistiske natur af SD kort modulet og højttaleren. Dog var dette for lavt til at kunne høres, især når motorerne også kørte. Vi blev nødt til at forstærke lyden. Derfor valgte vi at benytte en audio amplifier, til at forstærke lyden, stadig ved brug af SD kort modulet, til at afspille lyden fra. Den virkede, ved at vi førte 12 volt til amplifieren, i stedet for de tidligere 5 volt direkte til højttaleren. Dog var dette stadig ikke højt nok, så vi skiftede til at bruge DF playeren i stedet. Det var vores første tanke, at vi skulle bruge både DF-playeren og amplifieren, men det viste sig, at DF-playeren var mere end høj nok af sig selv. Derudover gjorde DF-playeren det også muligt at håndtere alt lyd fra et modul, eftersom den har en indbygget SD kort holder. Derfor kunne vi skille os af med både amplifieren og SD kort modulet.

Kvalitetssikring

Når der arbejdes på et længerevarende projekt, er det vigtigt løbende at teste diverse aspekter for at sikre kvaliteten af produktet. Projektgruppens egne holdninger og præferencer kan hurtigt styre et projekt på afveje hvis ikke der inddrages holdninger og observationer fra kilder uafhængige af projektet. Det er i øvrigt også meget relevant at de uafhængige kilder er en del af målgruppen, så projektet designs efter deres præferencer og behov.

Brugertest af robot højde

Inden produktionen af selve robotten gik i gang var højden en vigtig detalje at have på plads. For at spare på både vægt og materiale skulle robotten helst ikke være for høj, men den skulle heller ikke være så lav at det var ubehageligt at interagere med den. Der blev foretaget nogle hurtige quick and dirty brugertests omkring højden af robotten. Quick and dirty brugertest er hurtige uformelle brugertests, der kan bruges til hurtigt at få svar på et mindre spørgsmål. Fire elever af forskellige

¹ Lydklipet er taget fra Splice, som er et abonnentbaseret, copyright-fri lydbibliotek.

højder blev bedt om at røre ved en kasse i forskellige højder Figur 56. Højderne var som følger, 35 cm, 30 cm og 17 cm. 3 af testpersoner mellem højderne 165-175 cm fandt de 30 cm, som værende det mest behagelige, den sidste testperson som var 190 cm høj synes alle højderne var ubehagelige, dog var personen også øm efter fysisk træning fra dagen før. Robotten endte med at blive ca. 20 cm høj, hvilket er en del lavere end det brugertesten viste. Beslutningen om at den markante ændring i højde blev taget på baggrund af vægt. Robottens motorer skulle stadig kunne trække robotten uden problemer i en tilpas hastighed.



Figur 56 - Billede fra højde brugertest

Brugertest af samlet produkt

Grundet uforudsete tekniske fejl var der desværre ikke tid til at udføre en brugertest. En brugertest er dog stadig blevet planlagt, eftersom det helt fra starten af planlægningsfasen har været tiltænkt at udføre en. Optimalt set havde robotten været færdig flere uger før deadline, og der var blevet foretaget en længerevarende brugertest, for at teste hvordan elever og ansatte på Vid Gymnasier HTX i Grenaa ville behandle en robot med kunstige følelser og diverse interaktions muligheder. Det blev dog hurtigt tydeligt hvor ambitiøst et mål dette var. En mindre og mere overkommelig brugertest blev derfor planlagt senere.

Under planlægningen af brugertesten blev det hurtigt fastslået, at det skulle være en observationstest. En observationstest ville have været oplagt, da adgang til produktets tænkte miljø og brugere var frit tilgængeligt. En observationstest giver også bedre mulighed for at få upåvirket data fra testpersonerne, eftersom de interagerer med produktet i en virkelighedsnær situation. I modsætning til et interview eller spørgeskema har testpersonen ikke mulighed for at foretage sig store overvejelser omkring deres svar, og de er også mere tilbøjelige til at opføre sig naturligt, fremfor hvordan de forventer meningen. Under en observationstest er det vigtigt, at personer som foretager undersøgelsen, har mindst mulig involvering i testen: Dette skyldes at testpersonerne bør føle, at de ikke er til stede, da det kan ændre adfærden hos testpersonerne, og derved forringe kvaliteten af testens data. En observationstest ville derfor være oplagt, især da der var rig mulighed for at udføre observationstesten i det korrekte miljø og uden for meget indvirkning fra undersøgernes side (Hvass-Raun, Observationer, 2023).

Det præcise miljø hvori robotten skulle testes lå mellem tre muligheder. Det kunne enten være i et aflukket lokale, Science bygningen eller torvet. Science bygningen og torvet er begge relativt ens i og

med de åbne rum, det betyder dog at det kan være sværere at foretage testen, da den kan køre som den vil og har frit lege i et større område. I det åbne rum er der også større risiko for hvis noget går galt, at så kan det være svært at opdage og stoppe. Det er dog det mest realistiske miljø, testen kan udføres i, og også der hvor det ville være nemmest for undersøgerne at forblive i baggrunden og ikke påvirke testpersonerne. Det ville også forhåbentligt være muligt at undgå aktivt at inddrage testpersoner, men i stedet udelukkende satse på at elever og lærere selv tager initiativet. I det åbne rum vil der også være elever og lærere som går forbi robotten imens andre interagerer med den, og robotens opførsel i den situation ville blive tydelig, og om der er brug for ændringer i den sammenhæng. I et lukket lokale derimod er robotten meget mere begrænset i hvor langt den kan køre, det vil også i højere grad kræve at undersøgerne bliver involveret i processen, da der ikke naturligt vil være elever eller lærere gående forbi, som kunne interagere med robotten. I det lukkede rum ville det være nemmere at udføre testen, men på bekostning af kvaliteten af testens data.

Det åbne rum endte derfor som det valgte miljø, og mere specifikt, science bygningen. Science bygningen blev valgt fremfor torvet af hovedsageligt to grunde. Den første grund var at imens der var undervisning bliver torvet som regel brugt til stationært gruppearbejde, og blev derfor vurderet til ikke at tilskynde interaktion med en robot. Science bygningen derimod bliver også brugt til gruppearbejde og andet undervisning, men der er også en tendens til at der befinder sig flere nysgerrige elever i science bygningen eller elever som går igennem for at hente vand eller andet. Den anden grund er at torvet har rigtig mange borde og stole med ben, og robotens navigation kan godt overse dem, da de er ret tynde. Udover det er det også mere naturligt og sandsynligvis mindre forvirrende for lærere og elever at se en robot i science bygningen fremfor torvet, det er jo som regel der hvor de andre teknikprojekter også befinder sig.

Som tidligere nævnt var forhåbningen at robotten naturligt selv ville tiltrække testpersoner til sig. Gruppens medlemmer ville være spredt ud i science bygningen for at kunne observere alle interaktioner som måtte finde sted, uden at have robotten efterfulgt, hvilket også ville have gjort det tydeligere for testpersonerne at de bliver observeret. Efter en testperson ville have interageret med robotten, ville et af gruppens medlemmer, som havde overværet interaktionen, gøre sig til kende og forhåbentlig få tilladelse til at udføre et kort interview. Interviewet ville have spørgsmål som kommer omkring hvordan testpersonen opfattede interaktionen og hvad de følte fra robotten. Det er vigtigt at robotten rent praktisk er nem og forståelig at interagere med, men også hvordan den kan påvirke følelser og om der ligger en interesse i at behandle den dårligt. Spørgsmålene i interviewet kan ses i bilag 1.

Evaluering

For at evaluere projektet har vi på et gruppemøde diskuteret, hvad der gik godt i projektet, og hvad der gik mindre godt. Vi nåede frem til, at projektet overordnet gik godt - vi fik udviklet et produkt, som både var ambitiøst, passede til den overordnede case og problemformulering, og som interesserede og udfordrede os hver især i gruppen. Desuden var vi meget målrettede, og vi fik hurtigt startet op på projektet. Fra tidligere projekter havde vi erfaret, at vi skulle passe på, at projektets omfang ikke blev for stort, og det tog vi med i dette projekt. Dog var der stadig visse udfordringer; projektets originale omfang var stadigvæk for ambitiøst så nogle af de planlagte features, som f.eks. Wi-Fi i robotten til kommunikation med en server, blev skåret fra. Det udsprang mest af vores mangelfulde projektstyring, der også flere gange gav anledning til forvirring i gruppen, da det af og til blev svært at få en fornemmelse af, hvor langt vi hver især var med opgaver.

Af dette projekt har vi lært, at vi skal være bedre til projektstyring, herunder vores tidsplanlægning, kommunikation og løbende dokumentation i form af logbog samt skrivning af rapporten. Dette tog vi

med til USA-spilprojektet, og i eksamensprojektet vil vi også bruge denne erfaring. For at vi bedre kan planlægge vores arbejdsindsats, vil vi bruge Trello til at holde styr på opgaver og holde møder for hver arbejds-session for at sikre kommunikationen mellem gruppens medlemmer.

Konklusion

Under dette projekt har vi produceret en robot med menneskelignende karaktertræk med formål at besvare vores problemstilling: Hvordan skaber vi et fysisk interaktionsdesign, der undersøger empati for robotter med menneske- eller kæledyrslignende karaktertræk? Vi har desværre ikke været i stand til at udføre vores planlagte brugertest grundet implikationer med robotens hardware. Dog kan vi konkludere, at robotten skulle være i stand til at skabe empati hos brugeren for at undersøge vores problemstilling. Det har vi gjort ved at give robotten "nuttede" træk så som store øjne og fornemmelsen af et stort hoved. Derudover kan robotens tanker, som imiterer menneskers følelser, læses. Hvis vi havde mulighed for at udføre vores planlagte brugertest, kunne vi have givet et svar på, om vores projekt var lykkedes. Men fra gruppens synspunkt var produktet på baggrund af vores teori en succes.

Referencer

- Ansys. (2023, 03 12). *Ansys Workbench*. Retrieved from Ansys Workbench | Simulation Integration Platform: <https://www.ansys.com/products/ansys-workbench>
- Ansys. (2023, 03 12). *Quality Assurance*. Retrieved from Quality Assurance: Reliability & Accuracy | Ansys: <https://www.ansys.com/company-information/quality-assurance>
- Electro Schematics. (2023, 03 12). *HC-SR04: Datasheet, Specs, and More*. Retrieved from HC-SR04: Datasheet, Specs, and More | ElectroSchematics: <https://www.electroschematics.com/hc-sr04-datasheet/>
- ElectronicWings. (2023, 03 12). *ESP8266 WiFi Module Interfacing with Arduino UNO*. Retrieved from ElectronicWings: <https://www.electronicwings.com/arduino/esp8266-wifi-module-interfacing-with-arduino-uno>
- GitHub. (2023, Marts 12). *Websites for you and your projects*. Retrieved from GitHub: <https://pages.github.com/>
- Hvass-Raun, R. (2023, marts 9). Den iterative proces. In R. Hvass-Raun, *Digitalt Design og Udvikling* (pp. 7-10). Systime. Retrieved from Systime.dk: <https://ddu.systime.dk/?id=187>
- Hvass-Raun, R. (2023, marts 9). Observationer. In R. Hvass-Raun, *Digitalt Design og Udvikling* (pp. 97-98). Systime. Retrieved from Systime.dk: <https://ddu.systime.dk/?id=167>
- Jafari, M. (2023, 03 12). *GET ROLLING WITH OMNI-DIRECTIONAL WHEELS*. Retrieved from Get Rolling with Omni-Directional Wheels | Servo Magazine: <https://www.servomagazine.com/magazine/article/get-rolling-with-omni-directional-wheels>
- Jeff-Paredes. (2017, Juli 6). *Add WiFi to Arduino UNO*. Retrieved from hackster.io: <https://www.hackster.io/imjeffparedes/add-wifi-to-arduino-uno-663b9e>
- MIT Computer Science & Artificial Intelligence Laboratory. (2001, 04 04). *Robo-Rats Locomotion: Car-type Drive*. Retrieved from Robo: <https://groups.csail.mit.edu/drl/courses/cs54-2001s/car.html>
- Mori, M. (2012, Juni 12). *The Uncanny Valley: The Original Essay by Masahiro Mori*. Retrieved from IEEE Spectrum: <https://spectrum.ieee.org/the-uncanny-valley>
- Payne, M. (2021, 07 10). *FEA Simulation Mecanum Wheel*. Retrieved from FEA Simulation Mecanum Wheel | FEA Simulation Library: <https://fetchcfd.com/view-project/2408-#>
- photoelectric. (2009, 08 12). *Star Wars - R2D2 sounds*. Retrieved from Star Wars - R2D2 sounds - YouTube: <https://youtu.be/2-BKjnAgNgY>
- robocraze. (2022, 09 10). *Principal of ultrasonic sensor*. Retrieved from What is Ultrasonic Sensor: Working Principle & Applications: <https://robocraze.com/blogs/post/what-is-ultrasonic-sensor>
- Smith, D. G. (2018, August 28). *Why do we think tiny things are cute?* Retrieved from Popular Science: <https://www.popsoci.com/why-do-we-think-tiny-things-are-cute/>
- Trinh, T. (2018, 12 11). *Robot omni 3 wheel*. Retrieved from Robot omni 3 wheel | 3D CAD Model: Thang Trinh

vexrobotics. (2023, 03 12). *Creating a V5 Drivetrain*. Retrieved from Creating a V5 Drivetrain - VEX Library: <https://kb.vex.com/hc/en-us/articles/360035952771-Creating-a-V5-Drivetrain>

Wikipedia. (2023, 03 12). *Differential wheeled robot*. Retrieved from Differential wheeled robot - Wikipedia: https://en.wikipedia.org/wiki/Differential_wheeled_robot

Wikipedia Foundation. (2023, Marts 8). *Inverse-square law*. Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Inverse-square_law

Bilag

Bilag 1

Interview spørgsmål:

Hvad hedder robotten?

Nævn 3 ord, som beskriver robotten.

Hvad synes du om robottens hastighed?

Hvad synes du om robottens lyde?

På hvor mange måder kan man interagere med robotten?

Hvordan var robotten at interagere med?

Hvad fik robotten dig til at føle?

Hvordan behandlede du robotten?

Havde du lyst til at behandle den anderledes?

Hvad synes du om robotten som helhed?

Har du nogle yderligere bemærkninger?

Bilag 2

QuadMotorControl.ino:

```
// Includes

#include "Variables.h"
#include "DirectionEnum.h"
#include "DirectionChange.h"
#include "MotorSetup.h"
#include "SensorControl.h"
#include "SoundControl.h"
#include "AvoidanceSystem.h"
#include "SoundDetection.h"

#include "TouchSensor.h"
#include "DistanceMeasure.h"

#include "Screen.h"
#include "ActionChooser.h"

long waitTmp = millis();

// CODE

void setup() {
    Serial.begin(9600);
    ScreenSetup();
    lcd.setCursor(0, 0);
    lcd.print("Mood: " + String(mood) + " ");
    lcd.setCursor(0, 1);
    lcd.print("(^-^)");
    //SoundDetectionSetup();
    MotorSetup();
    SensorSetup();
    SoundPlayerSetup();
    showSensorInfo = false;
    Serial.begin(9600);
    ScreenSetup();
    lcd.setCursor(0, 0);
    lcd.print("Mood: " + String(mood) + " ");
    lcd.setCursor(0, 1);
    lcd.print("(^-^)");
    //SoundDetectionSetup();
    MotorSetup();
    SensorSetup();
    SoundPlayerSetup();
    showSensorInfo = false;
    delay(5000);
}

void loop(){
    if (millis() - ActionStart > ActionTime) {
        //lcd.print("Doing something");
        ActionStart = millis();
        ActionPicker();
    }

    if (millis() - LastMsgT > 3000){
```

```
        MsgPicker();
        LastMsgT = millis();
    }

    if (millis() - LastDrop > 10000){
        LastDrop = millis();
        if (mood > 0){
            mood = mood - 1;
            lcd.setCursor(0, 0);
            lcd.print("Mood: " + String(mood) + " ");
        }
    }
    SensorLoop();
    TouchLoop();
}

    if (millis() - ActionStart > ActionTime) {
        //lcd.print("Doing something");
        ActionStart = millis();
        ActionPicker();
    }

    if (millis() - LastMsgT > 3000){
        MsgPicker();
        LastMsgT = millis();
    }

    if (millis() - LastDrop > 10000){
        LastDrop = millis();
        if (mood > 0){
            mood = mood - 1;
            lcd.setCursor(0, 0);
            lcd.print("Mood: " + String(mood) + " ");
        }
    }
    SensorLoop();
    TouchLoop();
}
```

Bilag x

ActionChooser.h:

```
void opmearksomhed(){

    int ran;
    int minimumDistance = 25;

    SoundTrack angryTracks[] {Sur1, Sur2, Sur3};
    SoundTrack passivTracks[] {Passiv1, Passiv2, Passiv3};

    void efterforsk(){
        PlayRandomSound(passivTracks);
        lcd.clear();
        lcd.setCursor(2, 1);
        lcd.print("Efterforsk");
        MotionDirection();
    }

    void sang(){
        PlaySound(Sang1);
    }
}
```

```
        lcd.clear();
        lcd.setCursor(2, 1);
        lcd.print("Singing");
        ActionTime = 2000;
    }

void lur(){
    lcd.clear();
    lcd.setCursor(2, 1);
    lcd.print("Lur");
    ActionTime = 10000;
    // FindWall(true);
    PlaySound(Snork1);
}

void snurreRundt(){
    // SPiiiiinnnnnn
    lcd.clear();
    lcd.setCursor(2, 1);
    lcd.print("Spinning");
    int ran = random(1, 4);
    int LvsR = random(1, 3);
    int wait = ran * 1000;
    ActionTime = wait;
    if (LvsR = 1){ChangeDirection(RotateLeft, 255);}
    else {ChangeDirection(RotateRight, 255);}
}

void opmarksomhed(){
    lcd.clear();
    lcd.setCursor(2, 1);
    lcd.print("Wanting attention");
    PlayRandomSound(passivTracks);
    ActionTime = 7000;
    MotionDirection();
}

}

void selvhad(){
    PlayRandomSound(angryTracks);
    lcd.clear();
    lcd.setCursor(2, 1);
    lcd.print("Hating myself");
    ActionTime = 10000;
    PlayRandomSound(angryTracks);
    lcd.clear();
    lcd.setCursor(2, 1);
    lcd.print("Hating myself");
    ActionTime = 10000;
    //FindWall(false);
}

void vrede(){
    PlayRandomSound(angryTracks);
    lcd.clear();
    lcd.setCursor(2, 1);
    lcd.print("Angryyy");
```

```
ActionTime = 7000;
PlayRandomSound(angryTracks);
lcd.clear();
lcd.setCursor(2, 1);
lcd.print("Angryyy");
ActionTime = 7000;
MotionDirection();
}

void nysgerrig(){
    lcd.clear();
    lcd.setCursor(2, 1);
    lcd.print("Curious");
    ActionTime = 7000;
    //DetectSound(2000, 128);
    PlayRandomSound(passivTracks);
}

// ActiveAction
// ChangeDirection(dir, speed);

void ActionPicker (){

    if (mood >= 45 && avoiding == false && distance_N <= minimumDistance
    || distance_E <= minimumDistance || distance_S <= minimumDistance ||
    distance_W <= minimumDistance){
        ChangeDirection(Stop);
        Avoidance(minimumDistance);
    }

    if (mood > 85) {
        ran = random(0, 90);
        if (ran > 75) {efterforsk();}
        else if (ran > 60) {sang();}
        else if (ran > 55) {lur();}
        else if (ran > 40) {snurreRundt();}
        else if (ran > 30) {opmarksomhed();}
        else if (ran > 0) {nysgerrig();}
    }
    else if (mood > 70) {
        ran = random(0, 120);
        if (ran > 90) {efterforsk();}
        else if (ran > 85) {lur();}
        else if (ran > 65) {snurreRundt();}
        else if (ran > 45) {opmarksomhed();}
        else if (ran > 0) {nysgerrig();}
    }
    else if (mood > 45) {
        ran = random(0, 130);
        if (ran > 85) {efterforsk();}
        else if (ran > 75) {lur();}
        else if (ran > 45) {opmarksomhed();}
        else if (ran > 0) {nysgerrig();}
    }
    else if (mood > 15) {
        ran = random(0, 120);
        if (ran > 60) {efterforsk();}
        else if (ran > 45) {lur();}
    }
}
```

```
        else if (ran > 25) {opmarksomhed();}
        else if (ran > 0) {selvhad();}
    }
    else if (mood > 0) {
        ran = random(0, 100);
        if (ran > 50) {efterforsk();}
        else if (ran > 25) {selvhad();}
        else if (ran > 0) {vrede();}
    }
}

    if (mood >= 45 && avoiding == false && distance_N <= minimumDistance
|| distance_E <= minimumDistance || distance_S <= minimumDistance
|| distance_W <= minimumDistance){
    ChangeDirection(Stop);
    Avoidance(minimumDistance);
}

if (mood > 85) {
    ran = random(0, 90);
    if (ran > 75) {efterforsk();}
    else if (ran > 60) {sang();}
    else if (ran > 55) {lur();}
    else if (ran > 40) {snurreRundt();}
    else if (ran > 30) {opmarksomhed();}
    else if (ran > 0) {nysgerrig();}
}
if (mood > 85) {
    ran = random(0, 90);
    if (ran > 75) {efterforsk();}
    else if (ran > 60) {sang();}
    else if (ran > 55) {lur();}
    else if (ran > 40) {snurreRundt();}
    else if (ran > 30) {opmarksomhed();}
    else if (ran > 0) {nysgerrig();}
}
else if (mood > 70) {
    ran = random(0, 120);
    if (ran > 90) {efterforsk();}
    else if (ran > 85) {lur();}
    else if (ran > 65) {snurreRundt();}
    else if (ran > 45) {opmarksomhed();}
    else if (ran > 0) {nysgerrig();}
}
else if (mood > 45) {
    ran = random(0, 130);
    if (ran > 85) {efterforsk();}
    else if (ran > 75) {lur();}
    else if (ran > 45) {opmarksomhed();}
    else if (ran > 0) {nysgerrig();}
}
else if (mood > 15) {
    ran = random(0, 120);
    if (ran > 60) {efterforsk();}
    else if (ran > 45) {lur();}
    else if (ran > 25) {opmarksomhed();}
    else if (ran > 0) {selvhad();}
}
else if (mood > 0) {
    ran = random(0, 100);
    if (ran > 50) {efterforsk();}
    else if (ran > 25) {selvhad();}
```

```
        else if (ran > 0) {vrede();}  
    }  
}
```

Bilag x

AvoidanceSystem.h:

```
//Avoidance systemet er med til at sørge for at robotten ikke kører ind
i forhindringer, når vi ikke specifikt vil have det.
//Derfor er programmet programmeret til at starte dette recursive
blocking loop, som forhindre robotten i at kører ind i forhindringer.
//Det tidligere nævnte program er kun delvist lavet af mig, derfor er
det ikke med i afleveringen.
//Systemet modtager en minimums distance som parametre, som den bruger
til at tjekke distancerne til hver side af robotten.
void Avoidance(float minDistance){
    Serial.print("Avoiding - ");

    //Eftersom det er et blocking loop, bliver sensorværdierne ikke
    opdateret af det normale main loop, derfor tilføjes denne linje,
    //for at sikre at kun de mest nødvendige sensorværdier bliver brugt.
    Så robotten går i "fight or flight" mode, og benytter kun dens
    //mest bærbar funktioner.
    SensorLoop();

    //Herefter hardcodes alle blokeringsmulighederne i en række if-else
    statements. Dette gøres for at sikre at rækkefølgen bliver overholdt,
    //så den ikke begynder at reagere på en blokeringsmulighed fra øst
    (Og dermed går vest), hvis den både bliver blokeret fra øst og vest.
    //Derfor er de placeret så det mest relevante (4 retnings blokering)
    er først, derefter (3 retnings blokering), så (2 retningsblokering),
    //og til sidst (1 retningsblokering).
    //distance_N, distance_S, distance_E, og distance_W er forkortelser
    for den nuværende distance der er fra sensoren. _N = Nord, _S = Syd, _E
    = Øst, _W = Vest.
    if (distance_N < minDistance && distance_E < minDistance &&
    distance_S < minDistance && distance_W < minDistance){
        ChangeDirection(RotateLeft, 255);
        Serial.println("RotateLeft - 4");
    }
    else if (distance_N < minDistance && distance_E < minDistance &&
    distance_W < minDistance){
        ChangeDirection(South, 255);
        Serial.println("South - 3");
    }
    else if (distance_N < minDistance && distance_E < minDistance &&
    distance_S < minDistance){
        ChangeDirection(West, 255);
        Serial.println("West - 3");
    }
    else if (distance_N < minDistance && distance_S < minDistance &&
    distance_W < minDistance){
        ChangeDirection(East, 255);
        Serial.println("East - 3");
    }
    else if (distance_E < minDistance && distance_S < minDistance &&
    distance_W < minDistance){
        ChangeDirection(North, 255);
        Serial.println("North - 3");
    }
    else if (distance_N < minDistance && distance_E < minDistance){
        ChangeDirection(SouthWest, 255);
```

```
        Serial.println("SouthWest - 2");
    }
    else if (distance_N < minDistance && distance_W < minDistance){
        ChangeDirection(SouthEast, 255);
        Serial.println("SouthEast - 2");
    }
    else if (distance_S < minDistance && distance_W < minDistance){
        ChangeDirection(NorthEast, 255);
        Serial.println("NorthEast - 2");
    }
    else if (distance_E < minDistance && distance_S < minDistance){
        ChangeDirection(NorthWest, 255);
        Serial.println("NorthWest - 2");
    }
    else if (distance_N < minDistance && distance_S < minDistance){
        ChangeDirection(East, 255);
        Serial.println("East - 2");
    }
    else if (distance_E < minDistance && distance_W < minDistance){
        ChangeDirection(North, 255);
        Serial.println("North - 2");
    }
    else if (distance_N < minDistance){
        ChangeDirection(South, 255);
        Serial.println("South - 1");
    }
    else if (distance_E < minDistance){
        ChangeDirection(West, 255);
        Serial.println("West - 1");
    }
    else if (distance_S < minDistance){
        ChangeDirection(North, 255);
        Serial.println("North - 1");
    }
    else if (distance_W < minDistance){
        ChangeDirection(East, 255);
        Serial.println("East - 1");
    }
}

//Hvis så det viser sig at robotten ikke længere er i faresone,
bliver loopet stoppet igen, og robotten går tilbage til normal
    if (distance_N >= minDistance && distance_E >= minDistance &&
distance_S >= minDistance && distance_W >= minDistance){
        ChangeDirection(Stop);
        avoiding = false;
        return;
    }

//Derfor er denne linje sat til sidst, så loopet kan stoppes inden
det gentager sig selv.
    Avoidance(minDistance);
}

    Serial.print("Avoiding - ");

    //Eftersom det er et blocking loop, bliver sensorværdierne ikke
opdateret af det normale main loop, derfor tilføjes denne linje,
    //for at sikre at kun de mest nødvendige sensorværdier bliver brugt. Så
robotten går i "fight or flight" mode, og benytter kun dens
    //mest berygtede funktioner.
```

```
SensorLoop();

//Herefter hardcodes alle blokeringsmulighederne i en række if-else
statements. Dette gøres for at sikre at rækkefølgen bliver overholdt,
//så den ikke begynder at reagere på en blokeringsmulighed fra øst (Og
dermed går vest), hvis den både bliver blokeret fra øst og vest.
//Derfor er de placeret så det mest relevante (4 retnings blokering) er
først, derefter (3 retnings blokering), så (2 retningsblokering),
//og til sidst (1 retningsblokering).
//distance_N, distance_S, distance_E, og distance_W er forkortelser for
den nuværende distance der er fra sensoren. _N = Nord, _S = Syd, _E = Øst,
_W = Vest.
if (distance_N < minDistance && distance_E < minDistance && distance_S
< minDistance && distance_W < minDistance){
    ChangeDirection(RotateLeft, 255);
    Serial.println("RotateLeft - 4");
}
else if (distance_N < minDistance && distance_E < minDistance &&
distance_W < minDistance){
    ChangeDirection(South, 255);
    Serial.println("South - 3");
}
else if (distance_N < minDistance && distance_E < minDistance &&
distance_S < minDistance){
    ChangeDirection(West, 255);
    Serial.println("West - 3");
}
else if (distance_N < minDistance && distance_S < minDistance &&
distance_W < minDistance){
    ChangeDirection(East, 255);
    Serial.println("East - 3");
}
else if (distance_E < minDistance && distance_S < minDistance &&
distance_W < minDistance){
    ChangeDirection(North, 255);
    Serial.println("North - 3");
}
else if (distance_N < minDistance && distance_E < minDistance){
    ChangeDirection(SouthWest, 255);
    Serial.println("SouthWest - 2");
}
else if (distance_N < minDistance && distance_W < minDistance){
    ChangeDirection(SouthEast, 255);
    Serial.println("SouthEast - 2");
}
else if (distance_S < minDistance && distance_W < minDistance){
    ChangeDirection(NorthEast, 255);
    Serial.println("NorthEast - 2");
}
else if (distance_E < minDistance && distance_S < minDistance){
    ChangeDirection(NorthWest, 255);
    Serial.println("NorthWest - 2");
}
else if (distance_N < minDistance && distance_S < minDistance){
    ChangeDirection(East, 255);
    Serial.println("East - 2");
}
else if (distance_E < minDistance && distance_W < minDistance){
    ChangeDirection(North, 255);
    Serial.println("North - 2");
}
```

```
    else if (distance_N < minDistance){
        ChangeDirection(South, 255);
        Serial.println("South - 1");
    }
    else if (distance_E < minDistance){
        ChangeDirection(West, 255);
        Serial.println("West - 1");
    }
    else if (distance_S < minDistance){
        ChangeDirection(North, 255);
        Serial.println("North - 1");
    }
    else if (distance_W < minDistance){
        ChangeDirection(East, 255);
        Serial.println("East - 1");
    }
}

//Hvis så det viser sig at robotten ikke længere er i faresone, bliver
//loopet stoppet igen, og robotten går tilbage til normal
if (distance_N >= minDistance && distance_E >= minDistance &&
distance_S >= minDistance && distance_W >= minDistance){
    ChangeDirection(Stop);
    avoiding = false;
    return;
}

//Derfor er denne linje sat til sidst, så loopet kan stoppes inden det
//gentager sig selv.
Avoidance(minDistance);
}
```

Bilag 5

DirectionChange.h:

```
//Dette er en form for "Statemachine", som ændre retning og hastighed
af robotten, ud fra dens parametre

//Denne funktion bruger princippet "Overloading", eftersom der allerede
er en funktion der hedder "ChangeDirection",
//men denne funktion behøver ikke en hastigheds variable som parametre,
eftersom det er ligemeget, i det at den stopper robotten.

//Alle variablerne ift. pins bliver sat i et andet script (Som jeg ikke
har lavet 100% selv. Derfor er det ikke inkluderet).
//Alle "Direction" variablerne er lavet fra en enum (Her refererer jeg
til "DirectionEnum" scriptet.
void ChangeDirection(Direction newDirection){
    if (newDirection == Stop){

        //Eftersom alle case'ne indeholder det samme, men med forskellige
"HIGH" / "LOW" værdier, beskrives det kun her, og ikke resten af
scriptet.

        //Her sættes den foreste (T for top) venste (L for left) motor (M
for motor) til at stå stille.
        //_1 er den positive (røde) indgang til motoren
        //_2 er den negative (sorte) indgang til motoren
        //Eftersom det er en DC motor, kan retningen af motoren skiftes ud
fra om der sættes strøm til den positive eller den negative indgang.
        //Derfor, når begge indgange sættes til LOW, får den ikke strøm,
ergo. den stopper. Hvis _1 bliver sat til HIGH, når _2 er LOW, vil
motoren
        //dreje fremad. Hvis det er omvendt, altså _1 er LOW og _2 er HIGH,
vil motoren dreje baglæns.
        digitalWrite(TLM_1, LOW);
        digitalWrite(TLM_2, LOW);

        //Det samme sker her for (Top Right Motor)
        digitalWrite(TRM_1, LOW);
        digitalWrite(TRM_2, LOW);

        //Igen for (Back Left Motor)
        digitalWrite(BLM_1, LOW);
        digitalWrite(BLM_2, LOW);

        //Og til sidst for (Back Right Motor)
        digitalWrite(BRM_1, LOW);
        digitalWrite(BRM_2, LOW);

        //Til aller sidst, bliver hastigheden sat, ved at sætte hastigheden
til motor kontrol pins'ne. (Dette sker selvfølgelig med en varierende
variable i den
        //følgende funktion.
        analogWrite(TLEN, 0);
        analogWrite(TREN, 0);
        analogWrite(BLEN, 0);
        analogWrite(BREN, 0);
        if (newDirection == Stop){
```

```
//Eftersom alle case'ne indeholder det samme, men med forskellige
"HIGH" / "LOW" værdier, beskrives det kun her, og ikke resten af scriptet.

//Her sættes den foreste (T for top) venste (L for left) motor (M
for motor) til at stå stille.
//_1 er den positive (røde) indgang til motoren
//_2 er den negative (sorte) indgang til motoren
//Eftersom det er en DC motor, kan retningen af motoren skiftes ud
fra om der sættes strøm til den positive eller den negative indgang.
//Derfor, når begge indgange sættes til LOW, får den ikke strøm,
ergo. den stopper. Hvis _1 bliver sat til HIGH, når _2 er LOW, vil motoren
//dreje fremad. Hvis det er omvendt, altså _1 er LOW og _2 er HIGH,
vil motoren dreje baglæns.
digitalWrite(TLM_1, LOW);
digitalWrite(TLM_2, LOW);

//Det samme sker her for (Top Right Motor)
digitalWrite(TRM_1, LOW);
digitalWrite(TRM_2, LOW);

//Igen for (Back Left Motor)
digitalWrite(BLM_1, LOW);
digitalWrite(BLM_2, LOW);

//Og til sidst for (Back Right Motor)
digitalWrite(BRM_1, LOW);
digitalWrite(BRM_2, LOW);

//Til aller sidst, bliver hastigheden sat, ved at sætte hastigheden
til motor kontrol pins'ne. (Dette sker selvfølgelig med en varierende
variable i den
//følgende funktion.
analogWrite(TLEN, 0);
analogWrite(TREN, 0);
analogWrite(BLEN, 0);
analogWrite(BREN, 0);
}
}

//Her sker det samme, bortset fra at der modtages en hastighed med
retningen.
void ChangeDirection(Direction newDirection, int speedValue){
    //Staten bliver ændret ift. en switch statement (Hvilket er muligt,
eftersom der bliver gjort brug af en enum,
    //som jo egentligt er et tal fra 0 til 10 (I dette tilfælde)
    //Dette switch statement ændre motorens output til at være
overensstemmende med den retning den skal bevæge sig.
    switch (newDirection){
        case Stop:
            digitalWrite(TLM_1, LOW);
            digitalWrite(TLM_2, LOW);

            digitalWrite(TRM_1, LOW);
            digitalWrite(TRM_2, LOW);

            digitalWrite(BLM_1, LOW);
            digitalWrite(BLM_2, LOW);

            digitalWrite(BRM_1, LOW);
            digitalWrite(BRM_2, LOW);
```

```
    analogWrite(TLEN, 0);
    analogWrite(TREN, 0);
    analogWrite(BLEN, 0);
    analogWrite(BREN, 0);
    switch (newDirection){
        case Stop:
            digitalWrite(TLM_1, LOW);
            digitalWrite(TLM_2, LOW);

            digitalWrite(TRM_1, LOW);
            digitalWrite(TRM_2, LOW);

            digitalWrite(BLM_1, LOW);
            digitalWrite(BLM_2, LOW);

            digitalWrite(BRM_1, LOW);
            digitalWrite(BRM_2, LOW);

            analogWrite(TLEN, 0);
            analogWrite(TREN, 0);
            analogWrite(BLEN, 0);
            analogWrite(BREN, 0);
            break;
        case North:
            digitalWrite(TLM_1, HIGH);
            digitalWrite(TLM_2, LOW);

            digitalWrite(TRM_1, HIGH);
            digitalWrite(TRM_2, LOW);

            digitalWrite(BLM_1, HIGH);
            digitalWrite(BLM_2, LOW);

            digitalWrite(BRM_1, HIGH);
            digitalWrite(BRM_2, LOW);

            analogWrite(TLEN, speedValue);
            analogWrite(TREN, speedValue);
            analogWrite(BLEN, speedValue);
            analogWrite(BREN, speedValue);
            digitalWrite(TLM_1, HIGH);
            digitalWrite(TLM_2, LOW);

            digitalWrite(TRM_1, HIGH);
            digitalWrite(TRM_2, LOW);

            digitalWrite(BLM_1, HIGH);
            digitalWrite(BLM_2, LOW);

            digitalWrite(BRM_1, HIGH);
            digitalWrite(BRM_2, LOW);

            analogWrite(TLEN, speedValue);
            analogWrite(TREN, speedValue);
            analogWrite(BLEN, speedValue);
            analogWrite(BREN, speedValue);
            break;
        case South:
            digitalWrite(TLM_1, LOW);
```

```
digitalWrite(TLM_2, HIGH);

digitalWrite(TRM_1, LOW);
digitalWrite(TRM_2, HIGH);

digitalWrite(BLM_1, LOW);
digitalWrite(BLM_2, HIGH);

digitalWrite(BRM_1, LOW);
digitalWrite(BRM_2, HIGH);

analogWrite(TLEN, speedValue);
analogWrite(TREN, speedValue);
analogWrite(BLEN, speedValue);
analogWrite(BREN, speedValue);
digitalWrite(TLM_1, LOW);
digitalWrite(TLM_2, HIGH);

digitalWrite(TRM_1, LOW);
digitalWrite(TRM_2, HIGH);

digitalWrite(BLM_1, LOW);
digitalWrite(BLM_2, HIGH);

digitalWrite(BRM_1, LOW);
digitalWrite(BRM_2, HIGH);

analogWrite(TLEN, speedValue);
analogWrite(TREN, speedValue);
analogWrite(BLEN, speedValue);
analogWrite(BREN, speedValue);
break;
case East:
digitalWrite(TLM_1, HIGH);
digitalWrite(TLM_2, LOW);

digitalWrite(TRM_1, LOW);
digitalWrite(TRM_2, HIGH);

digitalWrite(BLM_1, LOW);
digitalWrite(BLM_2, HIGH);

digitalWrite(BRM_1, HIGH);
digitalWrite(BRM_2, LOW);

analogWrite(TLEN, speedValue);
analogWrite(TREN, speedValue);
analogWrite(BLEN, speedValue);
analogWrite(BREN, speedValue);
digitalWrite(TLM_1, HIGH);
digitalWrite(TLM_2, LOW);

digitalWrite(TRM_1, LOW);
digitalWrite(TRM_2, HIGH);

digitalWrite(BLM_1, LOW);
digitalWrite(BLM_2, HIGH);

digitalWrite(BRM_1, HIGH);
```

```
        digitalWrite(BRM_2, LOW);

        analogWrite(TLEN, speedValue);
        analogWrite(TREN, speedValue);
        analogWrite(BLEN, speedValue);
        analogWrite(BREN, speedValue);
        break;
case West:
    digitalWrite(TLM_1, LOW);
    digitalWrite(TLM_2, HIGH);

    digitalWrite(TRM_1, HIGH);
    digitalWrite(TRM_2, LOW);

    digitalWrite(BLM_1, HIGH);
    digitalWrite(BLM_2, LOW);

    digitalWrite(BRM_1, LOW);
    digitalWrite(BRM_2, HIGH);

    analogWrite(TLEN, speedValue);
    analogWrite(TREN, speedValue);
    analogWrite(BLEN, speedValue);
    analogWrite(BREN, speedValue);
    digitalWrite(TLM_1, LOW);
    digitalWrite(TLM_2, HIGH);

    digitalWrite(TRM_1, HIGH);
    digitalWrite(TRM_2, LOW);

    digitalWrite(BLM_1, HIGH);
    digitalWrite(BLM_2, LOW);

    digitalWrite(BRM_1, LOW);
    digitalWrite(BRM_2, HIGH);

    analogWrite(TLEN, speedValue);
    analogWrite(TREN, speedValue);
    analogWrite(BLEN, speedValue);
    analogWrite(BREN, speedValue);
    break;
case NorthEast:
    digitalWrite(TLM_1, HIGH);
    digitalWrite(TLM_2, LOW);

    digitalWrite(TRM_1, LOW);
    digitalWrite(TRM_2, LOW);

    digitalWrite(BLM_1, LOW);
    digitalWrite(BLM_2, LOW);

    digitalWrite(BRM_1, HIGH);
    digitalWrite(BRM_2, LOW);

    analogWrite(TLEN, speedValue);
    analogWrite(TREN, speedValue);
    analogWrite(BLEN, speedValue);
    analogWrite(BREN, speedValue);
    digitalWrite(TLM_1, HIGH);
```

```
        digitalWrite(TLM_2, LOW);

        digitalWrite(TRM_1, LOW);
        digitalWrite(TRM_2, LOW);

        digitalWrite(BLM_1, LOW);
        digitalWrite(BLM_2, LOW);

        digitalWrite(BRM_1, HIGH);
        digitalWrite(BRM_2, LOW);

        analogWrite(TLEN, speedValue);
        analogWrite(TREN, speedValue);
        analogWrite(BLEN, speedValue);
        analogWrite(BREN, speedValue);
        break;
case NorthWest:
    digitalWrite(TLM_1, LOW);
    digitalWrite(TLM_2, LOW);

    digitalWrite(TRM_1, HIGH);
    digitalWrite(TRM_2, LOW);

    digitalWrite(BLM_1, HIGH);
    digitalWrite(BLM_2, LOW);

    digitalWrite(BRM_1, LOW);
    digitalWrite(BRM_2, LOW);

    analogWrite(TLEN, speedValue);
    analogWrite(TREN, speedValue);
    analogWrite(BLEN, speedValue);
    analogWrite(BREN, speedValue);
    digitalWrite(TLM_1, LOW);
    digitalWrite(TLM_2, LOW);

    digitalWrite(TRM_1, HIGH);
    digitalWrite(TRM_2, LOW);

    digitalWrite(BLM_1, HIGH);
    digitalWrite(BLM_2, LOW);

    digitalWrite(BRM_1, LOW);
    digitalWrite(BRM_2, LOW);

    analogWrite(TLEN, speedValue);
    analogWrite(TREN, speedValue);
    analogWrite(BLEN, speedValue);
    analogWrite(BREN, speedValue);
    break;
case SouthEast:
    digitalWrite(TLM_1, LOW);
    digitalWrite(TLM_2, LOW);

    digitalWrite(TRM_1, LOW);
    digitalWrite(TRM_2, HIGH);

    digitalWrite(BLM_1, LOW);
    digitalWrite(BLM_2, HIGH);
```

```
digitalWrite(BRM_1, LOW);
digitalWrite(BRM_2, LOW);

analogWrite(TLEN, speedValue);
analogWrite(TREN, speedValue);
analogWrite(BLEN, speedValue);
analogWrite(BREN, speedValue);
digitalWrite(TLM_1, LOW);
digitalWrite(TLM_2, LOW);

digitalWrite(TRM_1, LOW);
digitalWrite(TRM_2, HIGH);

digitalWrite(BLM_1, LOW);
digitalWrite(BLM_2, HIGH);

digitalWrite(BRM_1, LOW);
digitalWrite(BRM_2, LOW);

analogWrite(TLEN, speedValue);
analogWrite(TREN, speedValue);
analogWrite(BLEN, speedValue);
analogWrite(BREN, speedValue);
break;
case SouthWest:
digitalWrite(TLM_1, LOW);
digitalWrite(TLM_2, HIGH);

digitalWrite(TRM_1, LOW);
digitalWrite(TRM_2, LOW);

digitalWrite(BLM_1, LOW);
digitalWrite(BLM_2, LOW);

digitalWrite(BRM_1, LOW);
digitalWrite(BRM_2, HIGH);

analogWrite(TLEN, speedValue);
analogWrite(TREN, speedValue);
analogWrite(BLEN, speedValue);
analogWrite(BREN, speedValue);
digitalWrite(TLM_1, LOW);
digitalWrite(TLM_2, HIGH);

digitalWrite(TRM_1, LOW);
digitalWrite(TRM_2, LOW);

digitalWrite(BLM_1, LOW);
digitalWrite(BLM_2, LOW);

digitalWrite(BRM_1, LOW);
digitalWrite(BRM_2, HIGH);

analogWrite(TLEN, speedValue);
analogWrite(TREN, speedValue);
analogWrite(BLEN, speedValue);
analogWrite(BREN, speedValue);
break;
case RotateLeft:
digitalWrite(TLM_1, LOW);
```

```
        digitalWrite(TLM_2, HIGH);

        digitalWrite(TRM_1, HIGH);
        digitalWrite(TRM_2, LOW);

        digitalWrite(BLM_1, LOW);
        digitalWrite(BLM_2, HIGH);

        digitalWrite(BRM_1, HIGH);
        digitalWrite(BRM_2, LOW);

        analogWrite(TLEN, speedValue);
        analogWrite(TREN, speedValue);
        analogWrite(BLEN, speedValue);
        analogWrite(BREN, speedValue);
        digitalWrite(TLM_1, LOW);
        digitalWrite(TLM_2, HIGH);

        digitalWrite(TRM_1, HIGH);
        digitalWrite(TRM_2, LOW);

        digitalWrite(BLM_1, LOW);
        digitalWrite(BLM_2, HIGH);

        digitalWrite(BRM_1, HIGH);
        digitalWrite(BRM_2, LOW);

        analogWrite(TLEN, speedValue);
        analogWrite(TREN, speedValue);
        analogWrite(BLEN, speedValue);
        analogWrite(BREN, speedValue);
        break;
    case RotateRight:
        digitalWrite(TLM_1, HIGH);
        digitalWrite(TLM_2, LOW);

        digitalWrite(TRM_1, LOW);
        digitalWrite(TRM_2, HIGH);

        digitalWrite(BLM_1, HIGH);
        digitalWrite(BLM_2, LOW);

        digitalWrite(BRM_1, LOW);
        digitalWrite(BRM_2, HIGH);

        analogWrite(TLEN, speedValue);
        analogWrite(TREN, speedValue);
        analogWrite(BLEN, speedValue);
        analogWrite(BREN, speedValue);
        break;
    }
}

        digitalWrite(TLM_1, HIGH);
        digitalWrite(TLM_2, LOW);

        digitalWrite(TRM_1, LOW);
```

```
        digitalWrite(TRM_2, HIGH);

        digitalWrite(BLM_1, HIGH);
        digitalWrite(BLM_2, LOW);

        digitalWrite(BRM_1, LOW);
        digitalWrite(BRM_2, HIGH);

        analogWrite(TLEN, speedValue);
        analogWrite(TREN, speedValue);
        analogWrite(BLEN, speedValue);
        analogWrite(BREN, speedValue);
        break;
    }
}
```

Bilag 6

DirectionEnum.h:

```
//En "liste" (enum) over alle de forskellige directions robotten kan
flytte sig.
enum Direction {
    Stop,
    North,
    South,
    East,
    West,
    NorthEast,
    NorthWest,
    SouthEast,
    SouthWest,
    RotateLeft,
    Stop,
    North,
    South,
    East,
    West,
    NorthEast,
    NorthWest,
    SouthEast,
    SouthWest,
    RotateLeft,
    RotateRight
};

//Eftersom enum'en "Direction" kun kan refereres til via. en int (0, 1,
2... osv.), er dette char array med til at give
//muligheden for at refererer til retningen med navn istedet for tal.
Dette gør det nemmere at skifte retningen,
//da det kan være svært at huske alle tallene.
#define IDNAME(name) #name
const char* directionNames[] = {
    IDNAME(Stop),
    IDNAME(North),
    IDNAME(South),
    IDNAME(East),
    IDNAME(West),
    IDNAME(NorthEast),
    IDNAME(NorthWest),
    IDNAME(SouthEast),
    IDNAME(SouthWest),
    IDNAME(RotateLeft),
    IDNAME(Stop),
    IDNAME(North),
    IDNAME(South),
    IDNAME(East),
    IDNAME(West),
    IDNAME(NorthEast),
    IDNAME(NorthWest),
    IDNAME(SouthEast),
    IDNAME(SouthWest),
    IDNAME(RotateLeft),
    IDNAME(RotateRight)
};
```


Bilag 7

DistanceMeasure.h:

```
float ultraMeasure(int TRIG_PIN, int ECHO_PIN) {
    distance_cm = GetDistance(TRIG_PIN, ECHO_PIN);
    distance_cm = GetDistance(TRIG_PIN, ECHO_PIN);
    return distance_cm;
}

float filter(float filterArray[], float rawArray[], bool comp) {
    // For normal
    if (!comp) {
        for (int i = 0; i < 10; i++){
            filterArray[i] = rawArray[i];
        }
    }
    else {
        for (int i = 10; i < 20; i++){
            filterArray[i - 10] = rawArray[i];
        }
    }
}

// 2. SORTING THE ARRAY IN ASCENDING ORDER
for (int i = 0; i < 9; i++) {
    for (int j = i + 1; j < 10; j++) {
        if (filterArray[i] > filterArray[j]) {
            float swap = filterArray[i];
            filterArray[i] = filterArray[j];
            filterArray[j] = swap;
        }
    }
}

double sum = 0;
for (int i = 3; i < 7; i++) {
    sum += filterArray[i];
}

}

// SORTING THE ARRAY IN ASCENDING ORDER
for (int i = 0; i < 9; i++) {
    for (int j = i + 1; j < 10; j++) {
        if (filterArray[i] > filterArray[j]) {
            float swap = filterArray[i];
            filterArray[i] = filterArray[j];
            filterArray[j] = swap;
        }
    }
}

double sum = 0;
for (int i = 3; i < 7; i++) {
    sum += filterArray[i];
}
```

```
    }

    return sum / 4;
}

void FindWall(bool Sleep) {
    distN = filter (filterArrayN, distRawN, false);
    distS = filter (filterArrayS, distRawS, false);
    distE = filter (filterArrayE, distRawE, false);
    distW = filter (filterArrayW, distRawW, false);

    int Dir;
    int Dist;

    if (distN < distS && distN < distE && distN < distW ) {
        ChangeDirection(North, 255);
        Dir = 1;
        ChangeDirection(North, 255);
        Dir = 1;
    }
    else if (distS < distN && distS < distE && distS < distW ) {
        ChangeDirection(South, 255);
        ChangeDirection(South, 255);
        Dir = -1;
    }
    else if (distE < distS && distE < distN && distE < distW ) {
        ChangeDirection(East, 255);
        Dir = 2;
        ChangeDirection(East, 255);
        Dir = 2;
    }
    else if (distW < distS && distW < distN && distW < distE ) {
        ChangeDirection(South, 255);
        ChangeDirection(South, 255);
        Dir = -2;
    }
}

while (Dist > 8){
    int TStart = millis();
    while (Dist < 15){}
    switch(Dir){
        case(1):
            Dist = distN;
            break;
        case(-1):
            Dist = distS;
            break;
        case(2):
            Dist = distE;
            break;
        case(-2):
            Dist = distW;
            break;
    }
    Dist = distN;
    break;
    case(-1):
        Dist = distS;
        break;
```

```
        case(2):
            Dist = distE;
            break;
        case(-2):
            Dist = distW;
            break;
    }
}
Dir = Dir *(-1);
if (Sleep == false) {
    switch(Dir){
        case(1):
            ChangeDirection(South, 255);
            long TStart = millis();
            while (millis()-TStart > 500){}
            ChangeDirection(North, 255);
            TStart = millis();
            while (millis()-TStart > 500){}
            break;
        case(-1):
            ChangeDirection(North, 255);
            TStart = millis();
            while (millis()-TStart > 500){}
            ChangeDirection(South, 255);
            TStart = millis();
            while (millis()-TStart > 500){}
            break;
        case(2):
            ChangeDirection(West, 255);
            TStart = millis();
            while (millis()-TStart > 500){}
            ChangeDirection(East, 255);
            TStart = millis();
            while (millis()-TStart > 500){}
            break;
        case(-2):
            ChangeDirection(East, 255);
            TStart = millis();
            while (millis()-TStart > 500){}
            ChangeDirection(West, 255);
            TStart = millis();
            while (millis()-TStart > 500){}
            break;
    }
}
switch(Dir){
    case(1):
        ChangeDirection(South, 255);
        long TStart = millis();
        while (millis()-TStart > 500){}
        ChangeDirection(North, 255);
        TStart = millis();
        while (millis()-TStart > 500){}
        break;
    case(-1):
        ChangeDirection(North, 255);
        TStart = millis();
        while (millis()-TStart > 500){}
        ChangeDirection(South, 255);
        TStart = millis();
        while (millis()-TStart > 500){}
```

```
        break;
    case(2):
        ChangeDirection(West, 255);
        TStart = millis();
        while (millis()-TStart > 500){}
        ChangeDirection(East, 255);
        TStart = millis();
        while (millis()-TStart > 500){}
        break;
    case(-2):
        ChangeDirection(East, 255);
        TStart = millis();
        while (millis()-TStart > 500){}
        ChangeDirection(West, 255);
        TStart = millis();
        while (millis()-TStart > 500){}
        break;
    }
}

void MotionDirection() {
    distN = filter (filterArrayN, distRawN, false);
    distCompN = filter (filterArrayCompN, distRawN, true);
    distS = filter (filterArrayS, distRawS, false);
    distCompS = filter (filterArrayCompS, distRawS, true);
    distE = filter (filterArrayE, distRawN, false);
    distCompE = filter (filterArrayCompE, distRawE, true);
    distW = filter (filterArrayW, distRawN, false);
    distCompW = filter (filterArrayCompW, distRawW, true);

    distDiffN = abs(distN - distCompN);
    distDiffS = abs(distS - distCompS);
    distDiffE = abs(distE - distCompE);
    distDiffW = abs(distW - distCompW);

    Serial.print("DistN: "); Serial.print(distDiffN);
    Serial.print(", DistS: "); Serial.print(distDiffS);
    Serial.print(", DistE: "); Serial.print(distDiffE);
    Serial.print(", DistW: "); Serial.println(distDiffW);

    if (distDiffN > 5) {ChangeDirection(North, 255);}
    if (distDiffS > 5) {ChangeDirection(South, 255);}
    if (distDiffE > 5) {ChangeDirection(East, 255);}
    distN = filter (filterArrayN, distRawN, false);
    distCompN = filter (filterArrayCompN, distRawN, true);
    distS = filter (filterArrayS, distRawS, false);
    distCompS = filter (filterArrayCompS, distRawS, true);
    distE = filter (filterArrayE, distRawN, false);
    distCompE = filter (filterArrayCompE, distRawE, true);
    distW = filter (filterArrayW, distRawN, false);
    distCompW = filter (filterArrayCompW, distRawW, true);

    distDiffN = abs(distN - distCompN);
    distDiffS = abs(distS - distCompS);
    distDiffE = abs(distE - distCompE);
    distDiffW = abs(distW - distCompW);

    Serial.print("DistN: "); Serial.print(distDiffN);
```

```
Serial.print(", DistS: "); Serial.print(distDiffS);  
Serial.print(", DistE: "); Serial.print(distDiffE);  
Serial.print(", DistW: "); Serial.println(distDiffW);  
  
if (distDiffN > 5) {ChangeDirection(North, 255);}  
if (distDiffS > 5) {ChangeDirection(South, 255);}  
if (distDiffE > 5) {ChangeDirection(East, 255);}  
if (distDiffW > 5) {ChangeDirection(West, 255);}  
}
```

Bilag 8

MotorSetup.h:

```
//Dette script er til for at rydde op i main scriptet, for at sikre at
det ikke bare bliver et langt script.
//Derfor tilføjes der funktioner som "FunktionNavn + Setup" eller
"FunktionsNavn + Loop", som derefter kan aktiveres
//i main setup'et eller main loop'et. Derfor er det også vigtigt at der
ikke tilføjes delays i nogen af scriptsene,
//eftesom dert ville sænke hele programmet.
void MotorSetup(){
    pinMode(TLEN, OUTPUT);
    pinMode(BLEN, OUTPUT);

    pinMode(TREN, OUTPUT);
    pinMode(BREN, OUTPUT);

    pinMode(TLM_1, OUTPUT);
    pinMode(TLM_2, OUTPUT);
    pinMode(TRM_1, OUTPUT);
    pinMode(TRM_2, OUTPUT);

    pinMode(BLM_1, OUTPUT);
    pinMode(BLM_2, OUTPUT);
    pinMode(BRM_1, OUTPUT);
    pinMode(BRM_2, OUTPUT);
}

    pinMode(TLEN, OUTPUT);
    pinMode(BLEN, OUTPUT);

    pinMode(TREN, OUTPUT);
    pinMode(BREN, OUTPUT);

    pinMode(TLM_1, OUTPUT);
    pinMode(TLM_2, OUTPUT);
    pinMode(TRM_1, OUTPUT);
    pinMode(TRM_2, OUTPUT);

    pinMode(BLM_1, OUTPUT);
    pinMode(BLM_2, OUTPUT);
    pinMode(BRM_1, OUTPUT);
    pinMode(BRM_2, OUTPUT);
}
```

Bilag 9

Screen.h:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 20, 4); // set the LCD address to 0x27 for
a 16 chars and 2 line display
// Er det 4 fordi det er 4-line display? og med 20 char?

String VeryHappyMsg[] = {"(*´▽`*)",
    "q(>V<q)",
    "Hej Til Alle Mine Venner",
    "Jubii !!!",
    "Jeg Elsker Livet På Skolen",
    "Hejsa (*´▽`*)",
    "Intet Kan Stoppe Mig!",
    "q(>V<q)",
    "Hej Til Alle Mine Venner",
    "Jubii !!!",
    "Jeg Elsker Livet På Skolen",
    "Hejsa (*´▽`*)",
    "Intet Kan Stoppe Mig!",
    "Jeg Er Uovervindelig!"};

String HappyMsg[] = {"Jeg Nyder At Udforske Skolen",
    " (^_^)",
    "Jeg Har Det Sjovt",
    "Hihi",
    "Mine Tandhjul Summer Af Glæde",
    "Skal Vi Til Fredagscafe",
    " (^_^)",
    "Jeg Har Det Sjovt",
    "Hihi",
    "Mine Tandhjul Summer Af Glæde",
    "Skal Vi Til Fredagscafe",
    "Hvad Betyder Mucki Bar"};

String NeutralMsg[] = {"Hej Med Dig",
    "Vil Du Age Mig På Hovedet ?",
    "Jeg Keder Mig Lidt",
    "Hvad Er Dette Sted",
    "Hvad Er Meningen Med Livet?",
    "Hvornaar Er Der Mad",
    "Mit Navn Er Sennep ... Hvad Er Dit ?"};

    "Vil Du Age Mig På Hovedet ?",
    "Jeg Keder Mig Lidt",
    "Hvad Er Dette Sted",
    "Hvad Er Meningen Med Livet?",
    "Hvornaar Er Der Mad",
    "Mit Navn Er Sennep ... Hvad Er Dit ?"};

String SadMsg[] = {"Jeg Savner Min Far, Mikkell Blom",
    "Hvorfor Er Jeg Her",
    "...Bip",
    "(O_O;)",
    "Jeg Er Traet Af At Foele Mig Uforstaaet",
    "Doer Herovre LOL",
```

```
"Det Her Sted Er Utrygt",
"Definitionen Af Kedsomhed:",
  "Hvorfor Er Jeg Her",
  "...Bip",
  "(O_O;)",
  "Jeg Er Traet Af At Foele Mig Uforstaaet",
  "Doer Herovre LOL",
  "Det Her Sted Er Utrygt",
  "Definitionen Af Kedsomhed:",
  "Hhhhhhhhhhhhhh"
};

String DepressedMsg[] = {"Jeg Hader Industrisamfundet.",
  "Jeg Foeler Mig Haabloes Og Ulykkelig.",
  "Jeg Kan Ikke Se En Positiv Fremtid.",
  "(X__X)",
  "(=⌒=)",
  "Fodr Mig, Smaa Dyr",
  "...",
  "Kan Man Doe Af Det Her",
  "Jeg Foeler Mig Haabloes Og Ulykkelig.",
  "Jeg Kan Ikke Se En Positiv Fremtid.",
  "(X__X)",
  "(=⌒=)",
  "Fodr Mig, Smaa Dyr",
  "...",
  "Kan Man Doe Af Det Her",
  "Frygt Mig, Menneske"
};

String RanMsg (String arr[], int arrL){
  int ran = random(0, arrL);
  return arr[ran];
}

int ran = random(0, arrL);
return arr[ran];
}

void MsgPicker(){
  String Msg = "";
  if (mood >85){Msg = RanMsg(VeryHappyMsg, 8);}
  else if (mood > 70) {Msg = RanMsg(HappyMsg, 7);}
  else if (mood > 45) {Msg = RanMsg(NeutralMsg, 7);}
  else if (mood > 15) {Msg = RanMsg(SadMsg, 9);}
  else if (mood >= 0) {Msg = RanMsg(DepressedMsg, 9);}
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Mood: " + String(mood) + " ");
  lcd.setCursor(0, 1);
  lcd.print(Msg);
}

void ScreenSetup()
{
  lcd.init(); // initialize the lcd\
  // Print a message to the LCD.
  lcd.backlight();
}
```

```
String Msg = "";  
if (mood > 85) {Msg = RanMsg(VeryHappyMsg, 8);}  
else if (mood > 70) {Msg = RanMsg(HappyMsg, 7);}  
else if (mood > 45) {Msg = RanMsg(NeutralMsg, 7);}  
else if (mood > 15) {Msg = RanMsg(SadMsg, 9);}  
else if (mood >= 0) {Msg = RanMsg(DepressedMsg, 9);}  
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("Mood: " + String(mood) + " ");  
lcd.setCursor(0, 1);  
lcd.print(Msg);  
}  
  
void ScreenSetup(){  
  lcd.init(); // initialize the lcd\  
  // Print a message to the LCD.  
  lcd.backlight();  
}
```

Bilag 10

SensorControl.h:

```
void SensorSetup() {
    pinMode(TRN, OUTPUT);
    pinMode(TRE, OUTPUT);
    pinMode(TRS, OUTPUT);
    pinMode(TRW, OUTPUT);
    pinMode(ECN, INPUT);
    pinMode(ECE, INPUT);
    pinMode(ECS, INPUT);
    pinMode(TRN, OUTPUT);
    pinMode(TRE, OUTPUT);
    pinMode(TRS, OUTPUT);
    pinMode(TRW, OUTPUT);
    pinMode(ECN, INPUT);
    pinMode(ECE, INPUT);
    pinMode(ECS, INPUT);
    pinMode(ECW, INPUT);
}

float GetDistance(int TriggerPin, int EchoPin) {
    digitalWrite(TriggerPin, LOW);
    delayMicroseconds(2);
    digitalWrite(TriggerPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(TriggerPin, LOW);
    float duration = pulseIn(EchoPin, HIGH);
    float distance = (duration * 0.0343) / 2;
    distance = constrain(distance, minDistance, maxDistance);
    digitalWrite(TriggerPin, LOW);
    delayMicroseconds(2);
    digitalWrite(TriggerPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(TriggerPin, LOW);
    float duration = pulseIn(EchoPin, HIGH);
    float distance = (duration * 0.0343) / 2;
    distance = constrain(distance, minDistance, maxDistance);
    return distance;
}

void SensorLoop() {
    distance_N = GetDistance(TRN, ECN);
    distance_E = GetDistance(TRE, ECE);
    distance_S = GetDistance(TRS, ECS);
    distance_W = GetDistance(TRW, ECW);

    // Measurements per 30th ms

    if (millis() - distTimer > 30) {
        distTimer = millis();
        distRawN[distCount] = distance_N;
        distRawS[distCount] = distance_S;
        distRawE[distCount] = distance_E;
        distRawW[distCount] = distance_W;

        distCount++;
        if (distCount == 20) {distCount = 0;}
    }
}
```

```
    }

    if (showSensorInfo) {
        Serial.print("N: ");
        Serial.print(distance_N);
        Serial.print(" - E: ");
        Serial.print(distance_E);
        Serial.print(" - S:");
        Serial.print(distance_S);
        Serial.print(" - W:");
        Serial.println(distance_W);
    }
}

distance_N = GetDistance(TRN, ECN);
distance_E = GetDistance(TRE, ECE);
distance_S = GetDistance(TRS, ECS);
distance_W = GetDistance(TRW, ECW);
// Measurements per 30th ms

if (millis()- distTimer > 30) {
    distTimer = millis();
    distRawN[distCount] = distance_N;
    distRawS[distCount] = distance_S;
    distRawE[distCount] = distance_E;
    distRawW[distCount] = distance_W;
    distCount ++;
    if (distCount == 20) {distCount = 0;}
}

if (showSensorInfo) {
    Serial.print("N: ");
    Serial.print(distance_N);
    Serial.print(" - E: ");
    Serial.print(distance_E);
    Serial.print(" - S:");
    Serial.print(distance_S);
    Serial.print(" - W:");
    Serial.println(distance_W);
}
}
```

Bilag 11

SoundControl.h:

```
#include "SoftwareSerial.h"
#include "DFRobotDFPlayerMini.h"

SoftwareSerial softwareSerial(10, 11);
DFRobotDFPlayerMini player;

void SoundPlayerSetup() {
    softwareSerial.begin(9600);

    if (player.begin(softwareSerial)) {
        player.volume(volumeLevel);
        player.play(1);
    } else {
        Serial.println("Connecting to DFPlayer Mini failed!");
        softwareSerial.begin(9600);

        if (player.begin(softwareSerial)) {
            player.volume(volumeLevel);
            player.play(1);
        }
        else {
            Serial.println("Connecting to DFPlayer Mini failed!");
        }
    }
}

enum SoundTrack {
    Nysgerrig1,
    Passiv1,
    Passiv2,
    Passiv3,
    Snork1,
    Sur1,
    Sur2,
    Sur3,
    Nysgerrig1,
    Passiv1,
    Passiv2,
    Passiv3,
    Snork1,
    Sur1,
    Sur2,
    Sur3,
    Sang1
};

#define IDNAME(name) #name
const char* soundTrackNames[] = {
    IDNAME(Nysgerrig1),
    IDNAME(Passiv1),
    IDNAME(Passiv2),
    IDNAME(Passiv3),
    IDNAME(Snork1),
    IDNAME(Sur1),
    IDNAME(Sur2),
    IDNAME(Sur3),
    IDNAME(Sang1)
```

```
    IDNAME(Nysgerrig1),
    IDNAME(Passiv1),
    IDNAME(Passiv2),
    IDNAME(Passiv3),
    IDNAME(Snork1),
    IDNAME(Sur1),
    IDNAME(Sur2),
    IDNAME(Sur3),
    IDNAME(Sang1)
};

void PlaySound(SoundTrack track){
    if (digitalRead(7) == 1){
        player.play(track + 1);
        if (digitalRead(7) == 1){
            player.play(track + 1);
        }
    }
}

void PlayRandomSound(SoundTrack tracks[]){
    if (digitalRead(7) == 1){
        int rnd = random(0, sizeof(tracks) / sizeof(int));
        player.play(tracks[rnd] + 1);
    }
    if (digitalRead(7) == 1){
        int rnd = random(0, sizeof(tracks) / sizeof(int));
        player.play(tracks[rnd] + 1);
    }
}
```

Bilag 12

SoundDetection.h:

```
#define Mic1 A1
#define Mic2 A0
#define Mic3 A3
#define Mic4 A2

void SoundDetectionSetup() {
    pinMode(Mic1, INPUT);
    pinMode(Mic2, INPUT);
    pinMode(Mic3, INPUT);
    pinMode(Mic4, INPUT);
    pinMode(Mic1, INPUT);
    pinMode(Mic2, INPUT);
    pinMode(Mic3, INPUT);
    pinMode(Mic4, INPUT);
    Serial.begin(9600); // initialize serial communications at 9600 bps:
}

//Sound detection function. Takes total time measuring in milliseconds,
//and how many measurements per mic. Be aware of limited memory in
//arduino.

void DetectSound(int totalTime, int arrL){

    int TNow = millis();

    int Arr1[arrL];
    int Arr2[arrL];
    int Arr3[arrL];
    int Arr4[arrL];
    int Delay = totalTime/arrL;

    int AvgN = 0;
    int AvgW = 0;
    int AvgS = 0;
    int AvgE = 0;

    for(int i = 0; i < arrL; i++) {
        Arr1[i] = analogRead(Mic1);
        Arr2[i] = analogRead(Mic2);
        Arr3[i] = analogRead(Mic3);
        Arr4[i] = analogRead(Mic4);
    }
    // while (millis() - TNow > Delay) {TNow = millis();}
    Arr1[i] = analogRead(Mic1);
    Arr2[i] = analogRead(Mic2);
    Arr3[i] = analogRead(Mic3);
    Arr4[i] = analogRead(Mic4);
}

// Filters (hopefully) all arrays at the same time.
for (int i = 0; i < 9; i++) {
    for (int j = i + 1; j < 10; j++) {
        if (Arr1[i] > Arr1[j]) {
            float swap = Arr1[i];
            Arr1[i] = Arr1[j];
            Arr1[j] = swap;
        }
    }
}
```

```
        Arr1[j] = swap;
    }
    if (Arr2[i] > Arr2[j]) {
        float swap = Arr2[i];
        Arr2[i] = Arr2[j];
        Arr2[j] = swap;
    }
    if (Arr3[i] > Arr3[j]) {
        float swap = Arr1[i];
        Arr3[i] = Arr3[j];
        Arr3[j] = swap;
    }
    if (Arr4[i] > Arr4[j]) {
        float swap = Arr4[i];
        Arr4[i] = Arr4[j];
        Arr4[j] = swap;
    }
    for (int j = i + 1; j < 10; j++) {
        if (Arr1[i] > Arr1[j]) {
            float swap = Arr1[i];
            Arr1[i] = Arr1[j];
            Arr1[j] = swap;
        }
        if (Arr2[i] > Arr2[j]) {
            float swap = Arr2[i];
            Arr2[i] = Arr2[j];
            Arr2[j] = swap;
        }
        if (Arr3[i] > Arr3[j]) {
            float swap = Arr1[i];
            Arr3[i] = Arr3[j];
            Arr3[j] = swap;
        }
        if (Arr4[i] > Arr4[j]) {
            float swap = Arr4[i];
            Arr4[i] = Arr4[j];
            Arr4[j] = swap;
        }
    }
}

// Designate middle avgs

double sum1 = 0;
double sum2 = 0;
double sum3 = 0;
double sum4 = 0;

for (int sample = arrL/2 - 2; sample < arrL/2 + 2; sample++) {
    sum1 += Arr1[sample];
    sum2 += Arr2[sample];
    sum3 += Arr3[sample];
    sum1 += Arr1[sample];
    sum2 += Arr2[sample];
    sum3 += Arr3[sample];
    sum4 += Arr4[sample];
}

AvgN = sum1 / 4;
```

```
AvgW = sum2 / 4;
AvgS = sum3 / 4;
AvgE = sum4 / 4;

int yComp;
int xComp;

int y;
int x;

// Comparison of averages to choose final direction

if (AvgN > AvgS) {yComp = AvgN; y = 1;}
else {yComp = AvgN; y = -1;}

if (AvgW > AvgS) {xComp = AvgW;x = -1;}
else {xComp = AvgE ;x = 1;}

if (yComp > xComp +2 ) {
    if (y == 1) { ChangeDirection(North, 255);}
    if (y == 1) { ChangeDirection(North, 255);}
    if (y == -1) { ChangeDirection(South, 255);}
}

else if (xComp > yComp + 2) {
    if (x == 1) {ChangeDirection(East, 255);}
    if (x == 1) {ChangeDirection(East, 255);}
    if (x == -1) {ChangeDirection(West, 255);}
}

else {
    if (y == 1 && x == 1){ ChangeDirection(NorthEast, 255);}
    if (y == 1 && x == -1) {ChangeDirection(NorthWest, 255);}
    if (y == -1 && x == 1) {ChangeDirection(SouthEast, 255);}
    if (y == -1 && x == -1) {ChangeDirection(SouthWest, 255);}
}

if (y == 1 && x == 1){ ChangeDirection(NorthEast, 255);}
if (y == 1 && x == -1) {ChangeDirection(NorthWest, 255);}
if (y == -1 && x == 1) {ChangeDirection(SouthEast, 255);}
if (y == -1 && x == -1) {ChangeDirection(SouthWest, 255);}

}
```

Bilag 13

TouchSensor.h:

```
int Left;
int Middle;
int Right;

const int pressureSensitivity = 200;

bool touchedLeft = false;
bool touchedRight = false;
bool touchedMiddle = false;

int timer = 1000;
long delayStart = 0;
bool delayRunning = false;

void CheckTime()
{
    if (delayRunning && ((millis() - delayStart) >= timer))
    {
        delayRunning = false;
        touchedLeft = false;
        touchedMiddle = false;
        touchedRight = false;
    }
}

void CheckTime() {
    if (delayRunning && ((millis() - delayStart) >= timer)) {
        delayRunning = false;
        touchedLeft = false;
        touchedMiddle = false;
        touchedRight = false;
    }
}

void TouchSetup(void) {
    Serial.begin(9600);
}

void TouchLoop(void) {
    CheckTime();
    Left = analogRead(A4);
    Middle = analogRead(A5);
    Right = analogRead(A6);

    if(Left > pressureSensitivity)
    {
        CheckTime();
        Left = analogRead(A4);
        Middle = analogRead(A5);
        Right = analogRead(A6);

        if(Left > pressureSensitivity) {
            //touchedLeft = true;
            if(touchedMiddle == true)
            {
                if(touchedRight == true)
                {
                    Serial.println("Sennep has been touched");
                }
            }
        }
    }
}
```

```
        mood = mood + 10;
        touchedLeft = false;
        touchedMiddle = false;
        touchedRight = false;
    }
    if (touchedRight == false)
    {
        touchedMiddle = false;
    }
}
if(touchedMiddle == false)
{
    touchedLeft = true;
    delayRunning = true;
    delayStart = millis();
}
if (touchedRight == true)
{
    if(touchedMiddle == false)
    {
        touchedRight = false;
    }
}
}
if(Middle > pressureSensitivity)
{
    //touchedMiddle = true;
    if(touchedRight == true)
    {
        touchedMiddle = true;
        delayStart = millis();

    }
    if(touchedLeft == true)
    {
        touchedMiddle = true;
        delayStart = millis();
    }
}
if(Right > pressureSensitivity)
{
    if(touchedMiddle == true)
    {
        if(touchedLeft == true)
        {
            Serial.println("Sennep has been touched");
            touchedLeft = false;
            touchedMiddle = false;
            touchedRight = false;
        }
        if (touchedLeft == false)
        {
            touchedMiddle = false;
        }
    }
}
if(touchedMiddle == false)
{
    touchedRight = true;
```

```
        delayStart = millis();
    }
    if (touchedLeft == true)
    {
        if(touchedMiddle == false)
        {
            touchedLeft = false;
        }
    }

    if(touchedMiddle == true){
        if(touchedRight == true){
            Serial.println("Sennep has been touched");
            mood = mood + 10;
            touchedLeft = false;
            touchedMiddle = false;
            touchedRight = false;
        }
        if (touchedRight == false){
            touchedMiddle = false;
        }
    }

    if(touchedMiddle == false){
        touchedLeft = true;
        delayRunning = true;
        delayStart = millis();
    }
    if (touchedRight == true){
        if(touchedMiddle == false){
            touchedRight = false;
        }
    }
}
if(Middle > pressureSensitivity){
    //touchedMiddle = true;
    if(touchedRight == true){
        touchedMiddle = true;
        delayStart = millis();
    }
    if(touchedLeft == true){
        touchedMiddle = true;
        delayStart = millis();
    }
}
if(Right > pressureSensitivity){
    if(touchedMiddle == true){
        if(touchedLeft == true){
            Serial.println("Sennep has been touched");
            touchedLeft = false;
            touchedMiddle = false;
            touchedRight = false;
        }
        if (touchedLeft == false){
            touchedMiddle = false;
        }
    }
    if(touchedMiddle == false){
        touchedRight = true;
        delayStart = millis();
    }
    if (touchedLeft == true){
```

```
        if(touchedMiddle == false){
            touchedLeft = false;
        }
    }
}

String GetPressureLevel(int pressure){
    String level;
    if (pressure < 10) {
        level = " - No pressure";
    } else if (pressure < 200) {
        level = " - Light touch";
    } else if (pressure < 500) {
        level = " - Light squeeze";
    } else if (pressure < 800) {
        level = " - Medium squeeze";
    } else {
        level = " - Big squeeze";
    }

    return level;
}

String level;
if (pressure < 10) {
    level = " - No pressure";
}
else if (pressure < 200) {
    level = " - Light touch";
}
else if (pressure < 500) {
    level = " - Light squeeze";
}
else if (pressure < 800) {
    level = " - Medium squeeze";
}
else {
    level = " - Big squeeze";
}

return level;
}
```

Bilag 14

Variables.h:

```
//BASICS
int mood = 50;
long LastMsgT;
long RunTime = 60*15*1000;
long LastDrop = millis();
int volumeLevel = 15;

// Action variables

long ActionStart = 0;
long ActionTime = 0;
bool Paused = false;

// Dist Measure

float filterArrayN[10];
float filterArrayS[10];
float filterArrayE[10];
float filterArrayW[10];

float filterArrayCompN[10];
float filterArrayCompS[10];
float filterArrayCompE[10];
float filterArrayCompW[10];

float distN;
float distS;
float distE;
float distW;

float distCompN;
float distCompS;
float distCompE;
float distCompW;

float distDiffN;
float distDiffS;
float distDiffE;
float distDiffW;

float distRawN[10];
float distRawS[10];
float distRawE[10];
float distRawW[10];

float distTimer;
int distCount = 0;

int distance_cm;

bool avoiding = false;

// VARIABLE SETUP
//
// M = Motor
```

```
// T = Top
// B = Bottom
// R = Right
// L = Left
// _1 = Roter Fremad
// _2 = Roter Bagud
//
// EN = Enable
//
// Left Side Motors

int TREN = 33;
int BREN = 27;

int TRM_1 = 29;
int TRM_2 = 30;

int BRM_1 = 25;
int BRM_2 = 26;

//Right Side Motors

int TLEN = 22;
int BLEN = 28;

int TLM_1 = 32;
int TLM_2 = 31;

int BLM_1 = 24;
int BLM_2 = 23;

// Other motor variables

int minMotorSpeed = 125;
int maxMotorSpeed = 255;

bool showDirectionInfo = false;
bool showJoystickInfo = false;
bool showSensorInfo = false;
bool joystickControlOn = false;

// Sensor Variables

// EC = Echo
// TR = Trigger
// N = North
// S = South
// E = East
// W = West

int ECN = 48;
int ECE = 46;
int ECS = 42;
int ECW = 44;

int TRN = 49;
int TRE = 47;
int TRS = 43;
```

```
int TRW = 45;

float distance_N;
float distance_E;
float distance_S;
float distance_W;

float minDistance = 10;
float maxDistance = 130;
```

[Bilag x](#)

TouchSensor.h